

Backend

사용자가 서버 사용을 신청하고 관리자가 승인 버튼을 누르면, 그 뒤로는 사람 손을 거치지 않고 Ubuntu 계정 생성부터 컨테이너 배정, 만료 알림까지 전부 자동으로 처리됩니다. Admin BE(Spring Boot)는 그 자동화를 실제로 돌리는 서버 이고, 이 섹션은 그 코드를 넘겨받아 유지보수해야 하는 사람을 위한 문서입니다.

GitHub 레포: [CSID-DGU/admin_be](#)

읽는 순서

처음 합류했다면 아래 순서대로 읽습니다. 뒤에 나오는 문서일수록 앞 문서에서 나온 개념을 이미 안다고 가정하고 설명 합니다.

순 서	문서	이 문서가 하는 일
1	시작	로컬에서 서버를 처음 띄우기까지 순서대로 안내
2	개요	이 시스템이 왜 필요했고 무엇으로 만들어졌는지 소개
3	시스템 아키텍 처	신청 버튼을 누른 순간부터 계정이 만들어지고 나중에 자동으로 정리되기까지, 실제로 일어나는 일을 순서대로 설명
4	도메인 설명	그 과정에서 다루는 데이터가 DB에 어떤 모양으로 저장되는지 설명 (ERD 포함)
5	코드 컨벤션	코드를 이런 방식으로 짜는 이유와 규칙 정리
6	핵심 설계 패턴	여러 도메인에서 반복해서 부딪히는 문제와 그 해결책 정리

참고자료

아래 문서들은 순서대로 읽기보다, 필요할 때 찾아보는 문서입니다.

문서	이 문서가 하는 일	이럴 때 읽으세요
인증·보안	로그인 이후 요청마다 사용자를 어떻게 증명하고 검증하는지 설명	로그인이 안 되거나 토큰 재발급 오류를 조사 할 때
외부 연동	BE가 계정·컨테이너 작업을 넘기는 Infra Server, 지표를 가져오는 Prometheus와의 통신 방식 정리	Infra Server와 통신하는 코드를 수정하거나 디버깅할 때
Redis 키 카 탈로그	BE가 쓰는 Redis 키를 패턴·TTL·저장과 삭제 시점까지 정리	Redis 관련 버그를 조사하거나 새 키를 추가 할 때
에러 코드 카 탈로그	API가 던지는 에러 59개를 상황별로 정리	API 에러 메시지의 원인을 조사하거나 새 에러 케이스를 추가할 때

문서	이 문서가 하는 일	이럴 때 읽으세요
운영 가이드	배포·장애 대응 등 운영 중 실행해야 할 절차 정리	배포하거나 운영 중 장애에 대응할 때

API 명세

컨트롤러에 붙은 Swagger 어노테이션에서 자동 생성되는 API 문서입니다. 요청·응답 형식을 코드 없이 바로 확인하고, 직접 호출도 해볼 수 있습니다.

항목	값
Swagger UI	http://210.94.179.18:30083/swagger-ui/index.html
Base URL	http://210.94.179.18:30083

시작

로컬 개발 환경을 처음 세팅할 때 따라야 할 순서를 안내합니다. IntelliJ 설치 확인부터 `application.yml` 준비, 로컬 DB 연결, 인코딩 설정, Swagger 확인까지 다룹니다. 신규 관리자는 이 문서부터 진행한 뒤 [개요](#)로 넘어갑니다.

1. 기술 스택 확인

- IDE: IntelliJ. 학생 계정이 있다면 Ultimate 버전 사용을 권장합니다.
- Java Version: Java 17. 로컬에 설치되어 있어야 합니다.
- Framework: Spring Boot. 별도 설정은 필요 없습니다.

2. 설정 파일(`application.yml`) 준비

보안상 설정 파일은 레포지토리에 포함되어 있지 않습니다.

- 관리자용 [Notion > BE 페이지 > application.yml 페이지](#)에서 `application.yml` 내용을 전체 복사합니다.
- 프로젝트 경로 `src/main/resources/` 에 `application.yml` 파일을 새로 만들고 내용을 붙여넣습니다.
- `application.yml` 을 수정할 때는 이 Notion 페이지도 함께 업데이트하고, PR 본문에도 수정 사실을 명시합니다.
- `application.yml` 은 `.gitignore` 에 등록되어 있어 커밋되지 않습니다.

3. 로컬 DB 및 프로파일 설정

로컬 MySQL에 `web-admin` 데이터베이스를 생성합니다. 사용자·비밀번호·포트는 자유롭게 설정합니다.

```
CREATE DATABASE `web-admin`;
```

`application.yml`의 `local` 프로파일 섹션에서 본인 로컬 DB의 `username`과 `password`가 맞는지 확인하고, 파일 맨 위의 `spring.profiles.active` 설정이 `local`로 되어 있는지 확인합니다.

```
server:
  port: 8080

spring:
  profiles:
    active: local #local, prod 중 선택
```

예시 설정입니다.

```
# 로컬 환경 설정(본인 로컬 환경에 맞게 수정)
---
api:
  base-url: http://localhost:8080

logging:
  level:
    org.hibernate.SQL: debug
    org.hibernate.type.descriptor.sql.BasicBinder: trace
spring:
  config:
    activate:
      on-profile: local
  datasource: # 본인 로컬 DB 설정
    url: jdbc:mysql://localhost:3306/web_admin?
serverTimezone=Asia/Seoul&useSSL=false&allowPublicKeyRetrieval=true
    username: root
    password: toni1234
```

4. 의존성 로드 (Gradle)

IntelliJ 우측의 Gradle 탭에서 Reload All Gradle Projects를 실행해 필요한 라이브러리를 모두 다운로드합니다.

5. 인코딩 설정

`message.properties`의 한글이 깨지지 않도록 아래 설정을 완료합니다.

- Settings(`Ctrl+Alt+S` 또는 `Cmd+,`) → Editor → File Encodings로 이동합니다.
- Properties Files (`*.properties`) 설정에서 Default encoding을 UTF-8로, Transparent native-to-ascii conversion을 체크합니다.
- Apply 후 확인합니다.

6. 애플리케이션 실행

`src/main/java/.../AdminBeApplication.java` (메인 클래스)를 찾아 Run으로 실행합니다. 콘솔에 에러 없이 로그가 올라오는지 확인합니다.

Mac OS에서 발생하는 netty 에러(`MacOSDnsServerAddressStreamProvider`)는 시스템 기본값으로 대체되어 로컬 실행에 문제가 없으므로 무시해도 됩니다.

7. Swagger 확인

Swagger는 FE 개발자와 API 명세를 공유하기 위한 문서입니다. `http://localhost:8080/swagger-ui/index.html#/` 에 접속해 확인합니다.

접속되지 않으면 `application.yml` 에 설정한 포트 번호와 8080이 일치하는지 확인합니다. 8080을 이미 사용 중이라면 해당 프로세스를 종료하거나 다른 포트를 사용합니다.

8. 문제가 발생했을 때

- 한글이 깨져서 보인다면: 5번 설정을 다시 확인하고, 이미 깨진 파일은 원본 텍스트로 다시 붙여넣습니다.
- DB 연결 에러가 난다면: `web-admin` 스키마 생성 여부와 비밀번호를 확인합니다. 해결되지 않으면 같은 포트에서 다른 DB가 실행 중인지 확인합니다.
- 그 외 문제는 팀원에게 문의합니다.

세팅을 마쳤다면 개요로 이동해 시스템 전체 구조를 확인합니다.

개요

코드를 보기 전에 이 시스템이 왜 만들어졌고 Admin BE가 그 안에서 무슨 몫을 맡는지부터 잡고 갑니다. 관리자의 반복 업무를 자동화한다는 목적, 그리고 그게 LAB·FARM이라는 서로 다른 서버 환경 때문에 왜 필요해졌는지를 먼저 다룹니다. 여기서 그림이 잡혀야 뒤에 나오는 개별 기능들이 왜 그렇게 설계됐는지 이해가 됩니다.

1. 시스템 소개 및 목적

동국대학교 AI LAB 자동화 시스템은 연구실 GPU 서버 자원 관리와 컨테이너 생성 자동화를 위한 시스템입니다. 기존에는 연구실 인원의 서버 사용 요청을 관리자가 수동으로 처리했으나, 신청 접수부터 계정 생성, 만료 처리까지 일련의 과정을 자동화하여 관리자 업무 부담을 줄이는 것을 목적으로 합니다.

연구실 인원(학생, 연구원)은 AI/ML 연구를 위해 GPU 자원이 필요합니다. 여러 인원이 한정된 수의 서버를 공유하므로, 각 사용자에게 독립된 컨테이너 환경을 제공합니다. 컨테이너는 K8s Pod로 생성되며, 사용자는 승인 후 발급받은 SSH 포트와 Jupyter 포트에 접속해 사용합니다.

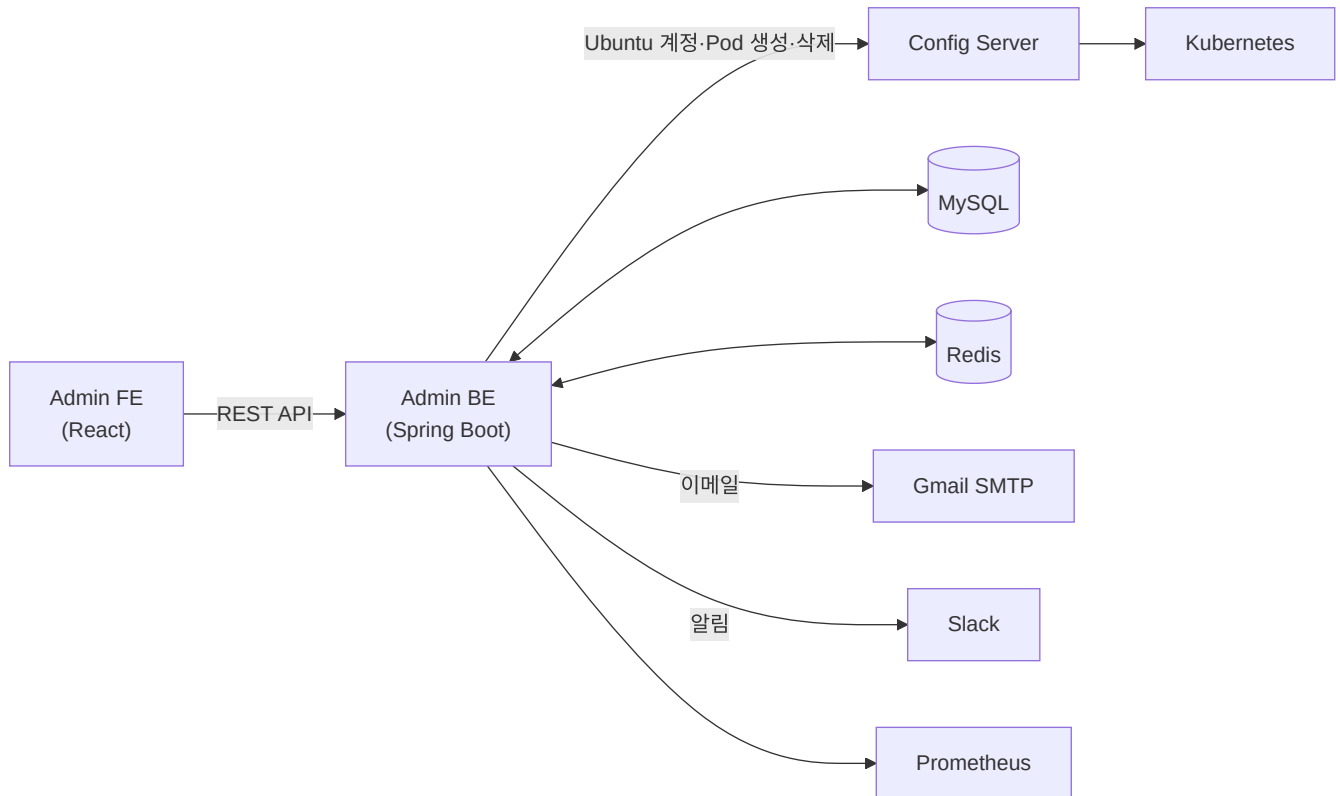
서버실에는 LAB과 FARM 두 개의 서버가 VLAN으로 분리되어 운영됩니다. 각 서버마다 탑재된 GPU 종류가 다르기 때문에, 사용자 신청 시 어느 서버에 컨테이너를 배치할지 적절히 오케스트레이션해야 합니다. 이처럼 이기종 서버 환경을 수동으로 관리하면 실수가 잦고 확장이 어렵기 때문에, 신청 접수부터 컨테이너 배치·회수까지 전 과정을 자동화하는 시스템이 필요합니다.

Admin BE는 이 자동화 시스템의 백엔드 서버로, 다음 역할을 담당합니다.

- 사용자/관리자 인증 및 권한 관리
- 서버 사용 신청 접수, 승인/거절 처리
- 승인 시 인프라 서버에 Ubuntu 계정 및 K8s 컨테이너 생성 요청
- 만료일 기반 컨테이너 자동 회수 및 사용자 알림 발송
- 장기 미접속 계정 자동 비활성화

본 문서는 Admin BE 인수인계 및 유지보수를 위한 기술 문서입니다.

2. 전체 구조



FE ↔ BE: 사용자·관리자의 모든 요청(신청, 승인, 조회 등)은 REST API로 들어옵니다.

BE → Infra: 승인 시 Infra Server에 Ubuntu 계정 생성과 K8s Pod 생성을 요청합니다. 만료·삭제 시에는 반대로 정리를 요청합니다.

BE → 알림: 승인, 만료 예고, 삭제 완료 등 주요 이벤트 발생 시 Gmail로 이메일을, Slack으로 알림을 보냅니다. Slack 알림은 Redis 큐를 통해 비동기로 발송합니다.

MySQL은 신청 내역, 사용자 정보, 포트 매핑 등 운영 데이터를 저장합니다. Redis는 JWT RefreshToken, Slack 알림 큐, 이메일 중복 방지에 사용합니다.

3. 기술 스택

- **Java 17**: 서버 사이드 애플리케이션 개발 언어
- **Spring Boot 3.5.3**: REST API 서버 구현, DI 컨테이너, 자동 설정
- **Spring Data JPA (Hibernate)**: DB 접근 추상화. 엔티티 매핑 및 쿼리 처리에 사용
- **MySQL 8.0.32**: 신청, 사용자, 컨테이너 상태 등 모든 영속 데이터 저장
- **Redis**: JWT RefreshToken 저장, Slack 알림 비동기 큐, 이메일 중복 방지(SETNX)
- **JWT (jjwt 0.11.5)**: 사용자 인증 토큰 생성 및 검증
- **Gradle 8.14.2**: 빌드 및 의존성 관리
- **Docker / Kubernetes / Helm (Fabric8 Kubernetes Client 6.10.0)**: 컨테이너 패키징 및 K8s 클러스터 배포

- **JavaMailSender (Gmail SMTP)**: 사용자 이메일 알림 발송
- **Slack Webhook / DM API**: 관리자 채널 알림 및 사용자 DM 발송
- **Swagger (springdoc-openapi 2.8.0)**: REST API 명세 자동 문서화

4. 서버 환경

서버는 LAB(210.94.179.18)과 FARM(210.94.179.19) 두 곳을 운영합니다. 두 서버는 VLAN으로 네트워크가 분리되어 있으며, 각 서버마다 탑재된 GPU 종류가 상이합니다. LAB 서버는 김지희 교수님 연구실에서 사용 중입니다.

Admin BE는 K8s 네임스페이스 `default` 에 배포되며, NodePort 30083을 통해 외부에서 접근합니다(Pod 내부 포트 :8080). MySQL과 Redis는 `ailab-be` 네임스페이스에 있습니다.

- **API Base URL**: `http://210.94.179.18:30083`
- **Swagger UI**: `http://210.94.179.18:30083/swagger-ui/index.html`

클러스터 내부 서비스 주소는 아래와 같습니다.

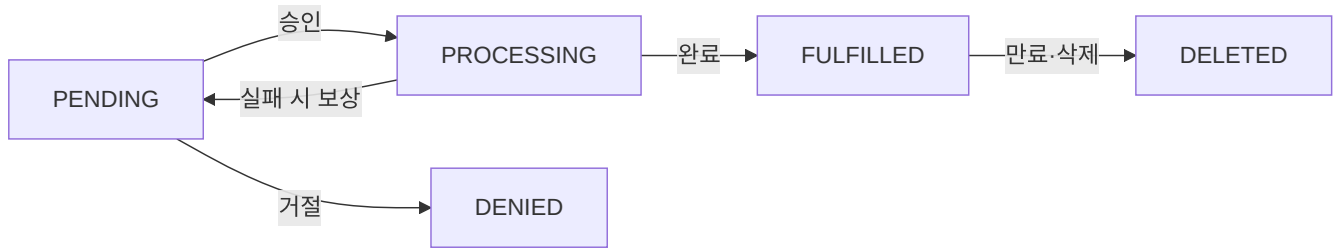
서비스	주소
MySQL	<code>my-mysql.ailab-be.svc.cluster.local:3306</code>
Redis	<code>my-redis-master.ailab-be.svc.cluster.local:6379</code>
Infra Server	<code>containerssh-config-service.ailab-infra.svc.cluster.local</code>

시스템 아키텍처

Request가 신청부터 승인·거절·만료까지 어떤 상태를 거치는지, 그리고 회원가입·로그인·신청·승인·삭제·만료 각 단계에서 FE·BE·MySQL·Redis·Infra Server·Gmail·Slack이 어떤 순서로 통신하는지를 시퀀스 다이어그램으로 정리합니다. 세 가지 스케줄러(만료 처리, 사용자 수명주기, Slack 큐 Worker)의 동작 방식과 BE·FE 내부 패키지 구조도 함께 다룹니다.

1. Request 생명주기

사용자가 서버 사용을 신청하면 Request가 생성되고, 관리자 승인 여부와 컨테이너 운영 상태에 따라 아래 상태를 따라 전이됩니다. PROCESSING은 승인 처리 중 외부 서버 호출이 실패했을 때 PENDING으로 되돌아오는 보상 처리를 위해 존재합니다.



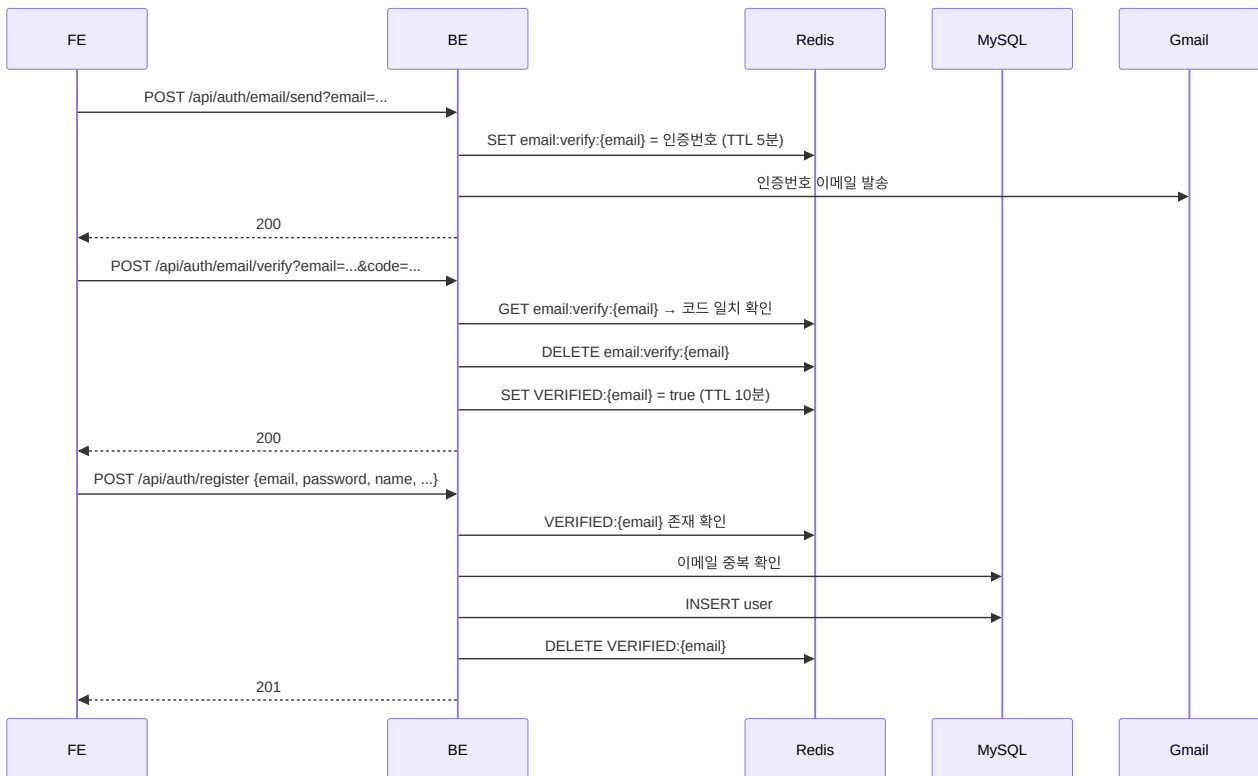
상태	설명
PENDING	신청 접수, 관리자 승인 대기
PROCESSING	승인 진행 중. 중복 승인 방지를 위해 사용
FULFILLED	승인 완료, 컨테이너 운영 중
DENIED	관리자 거절
DELETED	만료 또는 수동 삭제

2. 전체 흐름

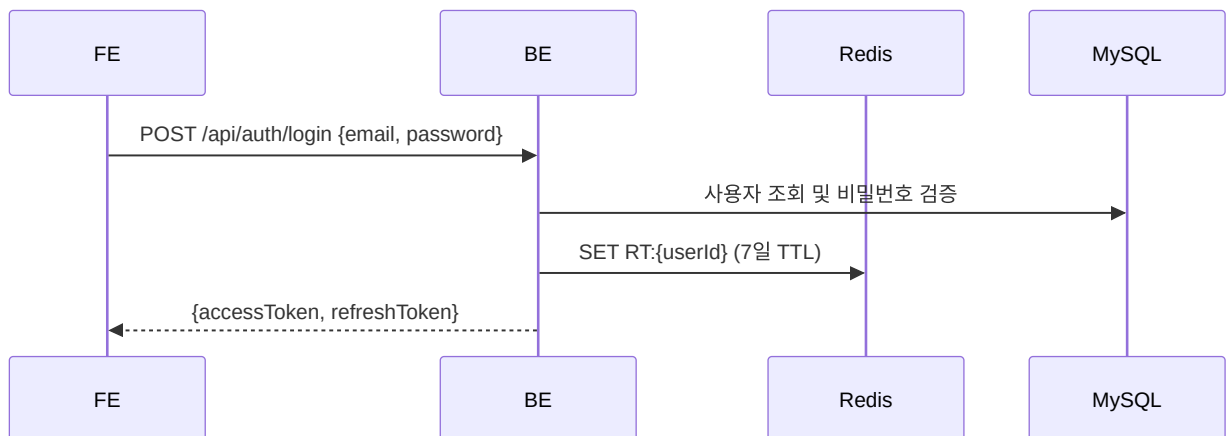
로그인부터 신청, 승인, 만료 처리까지 FE·BE·Infra Server 세 주체 간 통신 흐름입니다. 승인 과정에서는 Infra Server 에 Ubuntu 계정 생성과 Pod 생성을 순서대로 요청하며, 하나라도 실패하면 이전 단계를 되돌리는 보상 트랜잭션이 실행됩니다. 만료 처리는 스케줄러가 매일 08:00에 자동으로 수행합니다.

회원가입

이메일 인증 후 회원가입 순서입니다. 인증번호는 Redis에 5분 TTL로 저장하고, 인증 완료 상태(`VERIFIED:{email}`)는 10분간 유지됩니다. 회원가입 시 이 키가 있는지 확인하고, 저장 후 삭제합니다.

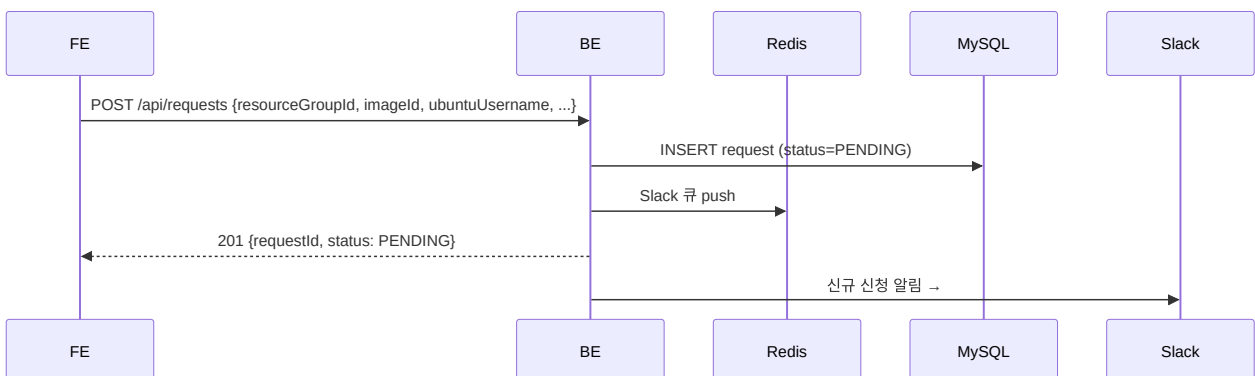


로그인

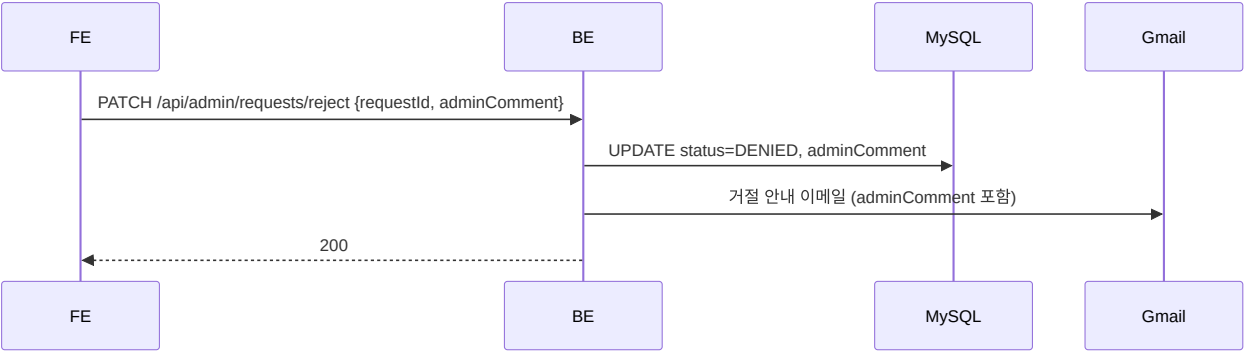


JWT 토큰 구조, 재발급, 로그아웃 흐름의 상세는 [인증·보안](#)을 참고합니다.

서버 사용 신청

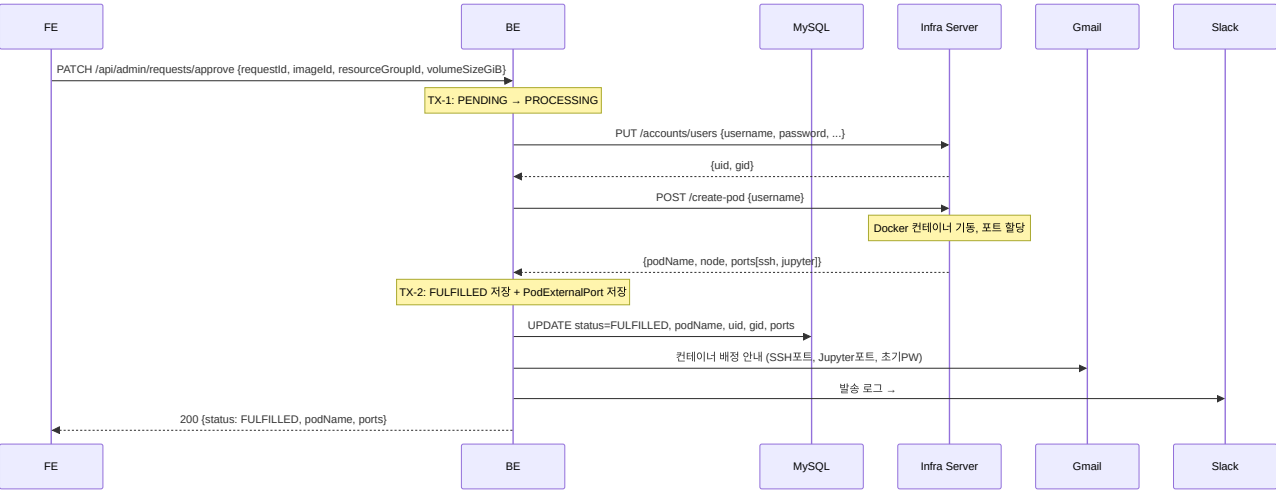


관리자 거절



이메일 발송은 try-catch로 감싸져 있어, 발송이 실패해도 거절 처리 자체는 그대로 완료됩니다. 실패 시 `AdminRequestCommandService` 에 WARN 로그만 남습니다.

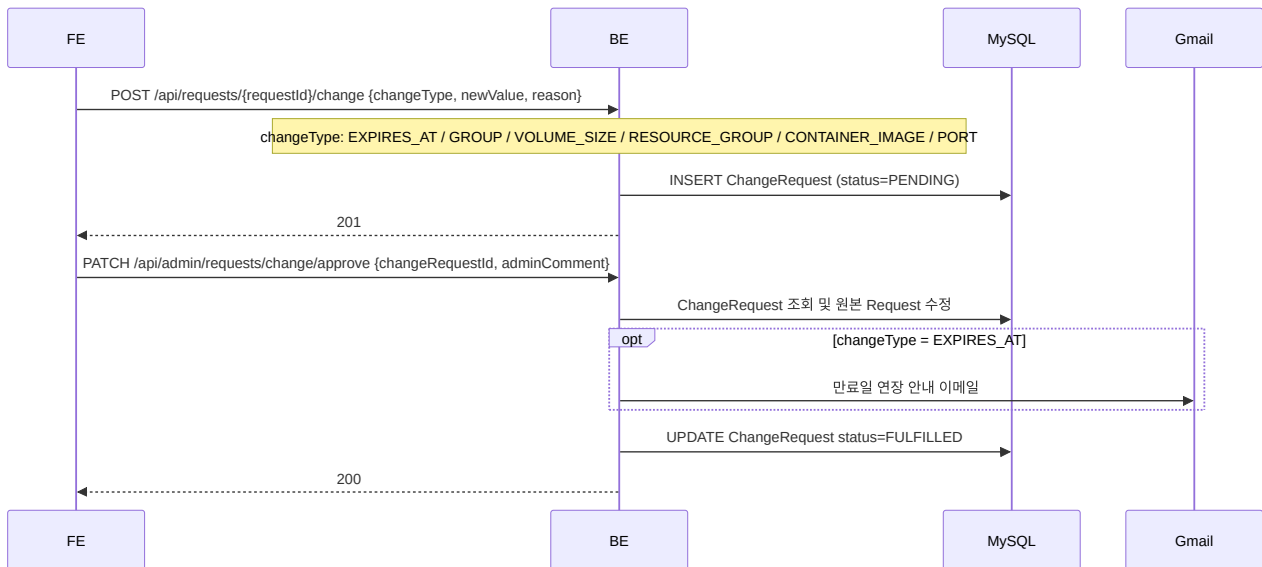
관리자 승인



Infra Server API 구조와 WebClient 타임아웃 설정 상세는 [외부 연동](#)을 참고합니다.

변경 요청 (만료일 연장·그룹 변경 등)

FULFILLED 상태인 컨테이너에 대해 사용자가 만료일 연장, 그룹 변경, 볼륨 크기 변경 등을 요청할 수 있습니다. 변경 요청은 별도 `ChangeRequest` 테이블에 PENDING으로 저장되고, 관리자가 승인하면 원본 `Request`에 반영됩니다. 만료일 연장이 승인된 경우에만 이메일이 발송됩니다.

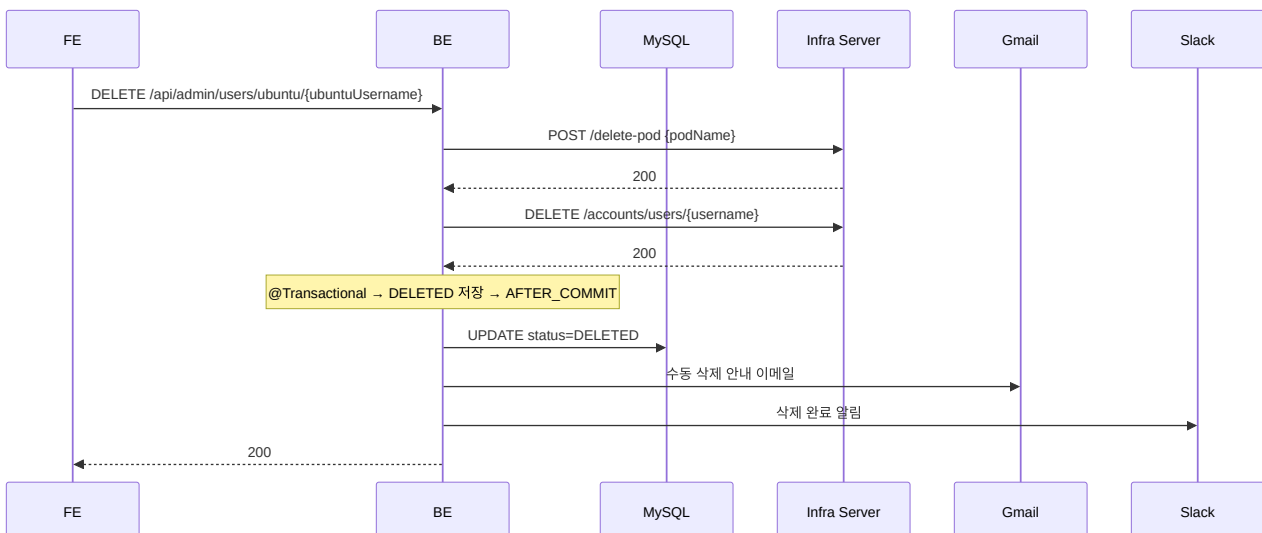


변경 요청이 거절되는 경우(`PATCH /api/admin/requests/change/reject`)에는 changeType과 무관하게 항상 거절 안내 이메일이 발송됩니다. 승인 흐름과 달리 EXPIRES_AT 조건이 없습니다.

관리자 수동 삭제

FULFILLED 상태인 컨테이너를 관리자가 직접 삭제합니다. 스케줄러 자동 삭제와 같은 Infra 호출·이메일 흐름을 따르지만, 즉시 처리된다는 점이 다릅니다.

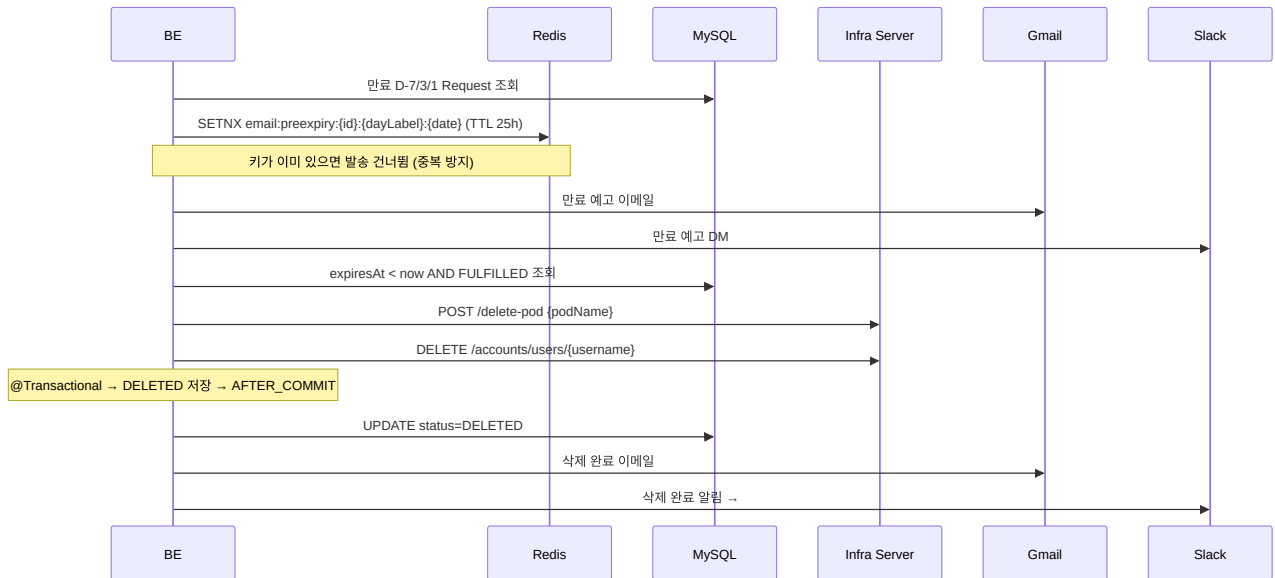
삭제 API는 두 가지입니다. 컨테이너 한 개만 지정해서 지우는 `DELETE /api/admin/users/ubuntu/{ubuntuUsername}` 과, 사용자 단위로 그 사람이 가진 FULFILLED 컨테이너를 모두 지우는 `DELETE /api/admin/users/{id}` (`AdminUserService.deleteUser()`)입니다. 아래 시퀀스는 전자(단일 컨테이너)를 기준으로 합니다. 후자는 같은 삭제 로직을 사용자의 Request 개수만큼 반복하며, 컨테이너별로 삭제 안내 이메일을 개별 발송합니다.



Pod 삭제나 Ubuntu 계정 삭제가 실패해도 DB는 `DELETED` 로 처리되며, Infra 리소스는 수동 정리가 필요합니다. 보상 트랜잭션은 승인 흐름에만 적용됩니다.

만료 처리 (매일 08:00 KST, 스케줄러)

만료 예고 이메일은 SETNX로 중복 발송을 막습니다. 스케줄러가 매일 실행되기 때문에, 같은 날 같은 사용자에게 D-7 이메일이 두 번 나가는 상황이 생길 수 있습니다. 발송 전에 `email:preexpiry:{id}:{dayLabel}:{date}` 키를 Redis에 SETNX로 쓰고, 이미 키가 존재하면 발송을 건너뛵니다. TTL은 25시간이라 하루가 지나면 자동으로 만료됩니다.



3. 스케줄러

BE 내부에 세 가지 스케줄러가 상시 동작합니다.

스케줄러	실행	역할
<code>RequestSchedulerService</code>	매일 08:00	D-7/3/1 만료 예고 알림 발송, 만료된 Request 삭제 처리
<code>UserSchedulerService</code>	매일 09:00	3개월 미접속 경고 → 비활성화(Soft Delete, 영구 보존)
<code>SlackNotificationWorker</code>	1초 간격	<code>slack:notification:queue</code> 에서 메시지를 꺼내 Slack API로 발송, 최대 3회 재시도
<code>InfraSlackNotificationWorker</code>	1초 간격	<code>slack:infra:notification:queue</code> 전용 Worker. 백엔드 알림 큐와 완전히 독립 운영, 최대 3회 재시도

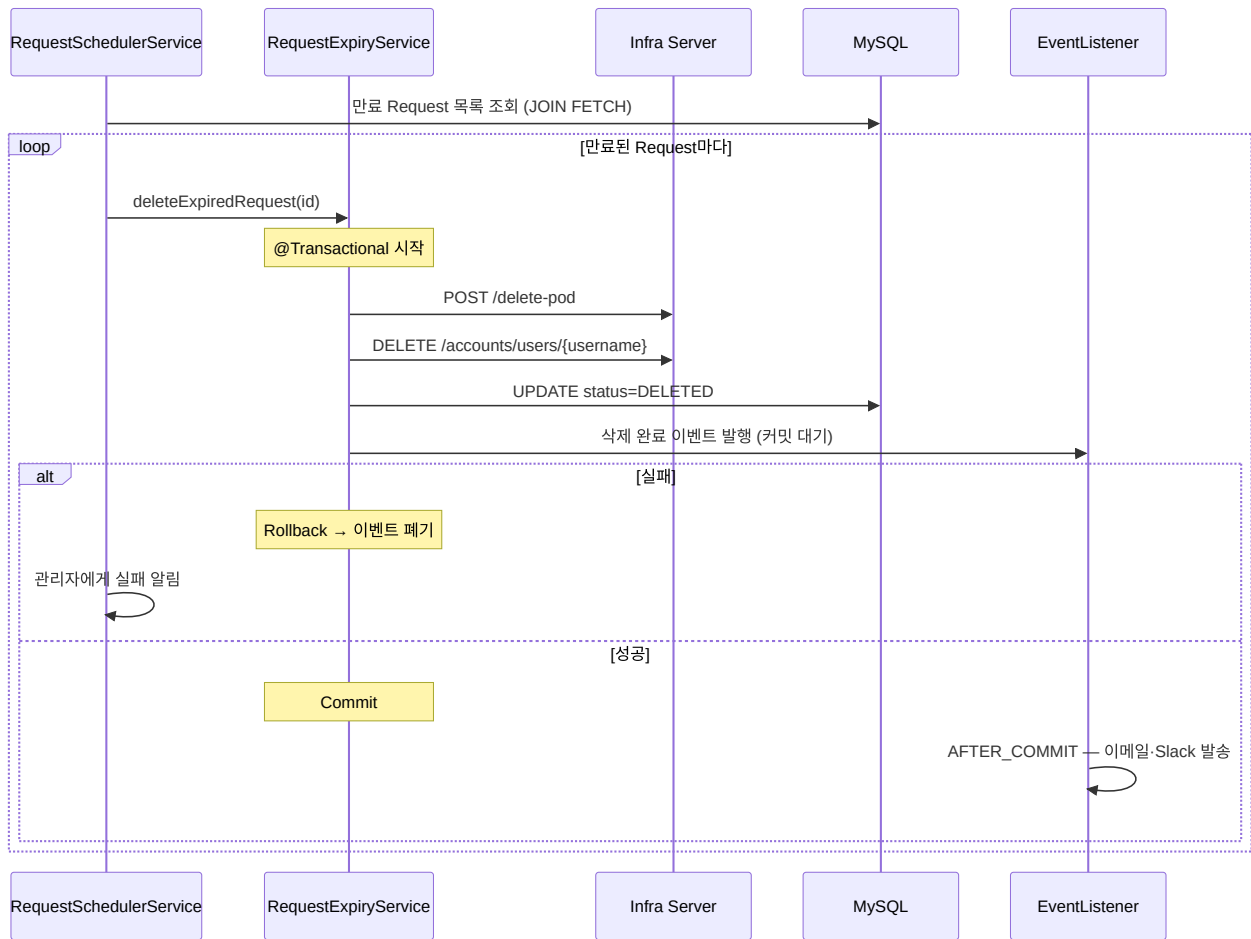
RequestSchedulerService

만료된 컨테이너를 삭제하는 흐름에서 주의할 점이 세 가지 있습니다.

첫째, 만료 대상 조회 시 `JOIN FETCH` 로 User·ResourceGroup을 한 번에 가져옵니다. 단순 조회 후 알림 발송 단계에서 연관 엔티티를 건당 추가 조회하는 N+1 문제를 막기 위해서입니다.

둘째, 삭제 메서드 호출 시 같은 클래스 내에서 `this.delete()` 로 호출하면 Spring AOP 프록시를 우회해 `@Transactional` 이 적용되지 않습니다. 이를 피하기 위해 `@Transactional` 메서드를 별도 클래스 (`RequestExpiryService`)로 분리하고, 해당 Bean을 주입받아 호출합니다.

셋째, 삭제 완료 알림은 `@TransactionalEventListener(phase = AFTER_COMMIT)` 를 통해 DB 커밋이 확정된 후에만 발송됩니다. 트랜잭션이 롤백되면 이벤트도 함께 폐기되므로, DB에 반영되지 않은 삭제에 대한 알림이 발송되는 상황을 방지합니다.



AFTER_COMMIT과 Self-Injection 패턴의 배경은 [핵심 설계 패턴 1·2번](#)을 참고합니다.

UserSchedulerService

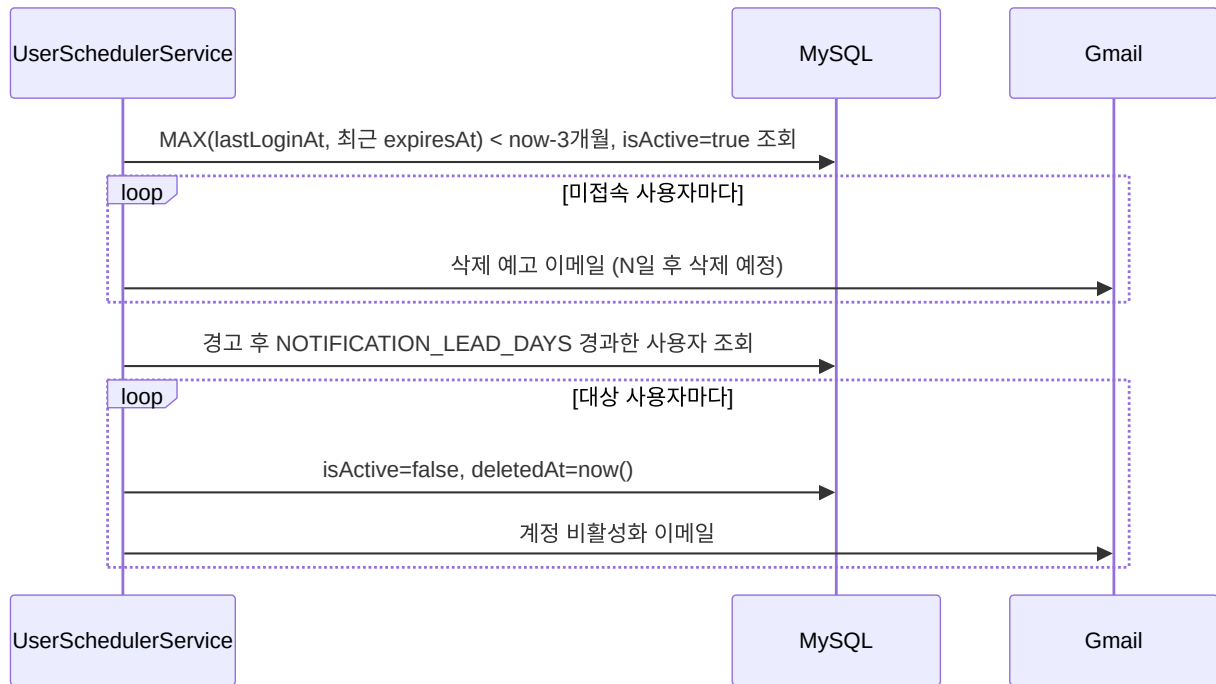
사용자 수명주기를 관리합니다. 매일 09:00에 실행되며 두 단계를 처리합니다. 이전에는 Soft Delete 1년 경과 후 DB row를 완전히 삭제하는 Hard Delete 단계가 있었으나, 이력 보존을 위해 제거되었습니다. 비활성화된 계정은 Soft Delete 상태로 영구 보존됩니다.

1단계 — 장기 미접속 경고

마지막 로그인 시각(`lastLoginAt`)과 해당 사용자의 가장 최근 컨테이너 만료일(`expiresAt`) 중 더 늦은 날짜를 기준으로, 3개월(`INACTIVE_MONTHS=3`) 이상 경과한 활성 사용자(`isActive=true`)를 조회해 삭제 예고 이메일을 발송합니다. 컨테이너가 활성 상태라면 로그인이 없어도 삭제 대상에서 제외됩니다. 이 시점에서는 계정을 삭제하지 않고 경고만 합니다.

2단계 — Soft Delete

경고 이메일을 받은 사용자가 일정 기간(`NOTIFICATION_LEAD_DAYS=8`) 내에 로그인하지 않으면 계정을 비활성화합니다. `isActive=false`, `deletedAt=now()` 로 설정하고 비활성화 이메일을 발송합니다. 이후 별도의 삭제 처리는 없습니다.



각 사용자 처리는 독립 트랜잭션으로 실행됩니다(`UserLifecycleTransactionalService`에 위임). 특정 사용자 처리 실패가 다른 사용자 처리에 영향을 주지 않습니다.

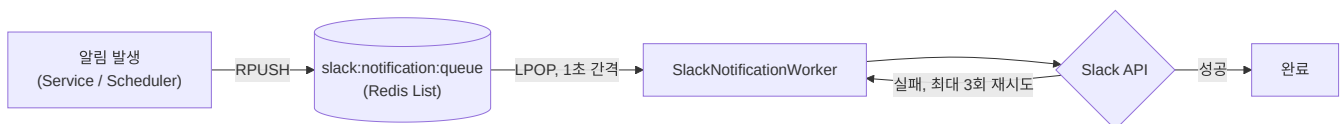
SlackNotificationWorker / InfraSlackNotificationWorker

승인, 삭제 등 비즈니스 로직이 Slack API 응답을 기다리며 블로킹되는 것을 막기 위해 Redis 큐를 사이에 둡니다. 알림이 발생하면 메시지를 큐에 즉시 적재하고, 전용 Worker가 1초마다 꺼내 Slack API를 호출합니다. 실패 시 최대 3회 재시도합니다.

일반 비즈니스 알림(`slack:notification:queue`)과 Infra 관련 알림(`slack:infra:notification:queue`)은 큐뿐 아니라 이를 소비하는 Worker 클래스(`SlackNotificationWorker`, `InfraSlackNotificationWorker`)도 완전히 분리되어 있습니다. 한쪽 큐가 밀려도 다른 쪽 발송에는 영향을 주지 않습니다.

1초 간격으로 설정한 이유는 Slack Webhook API의 rate limit이 채널당 초당 1건이기 때문입니다. 더 빠르게 폴링하면 제한에 걸려 메시지가 누락될 수 있습니다.

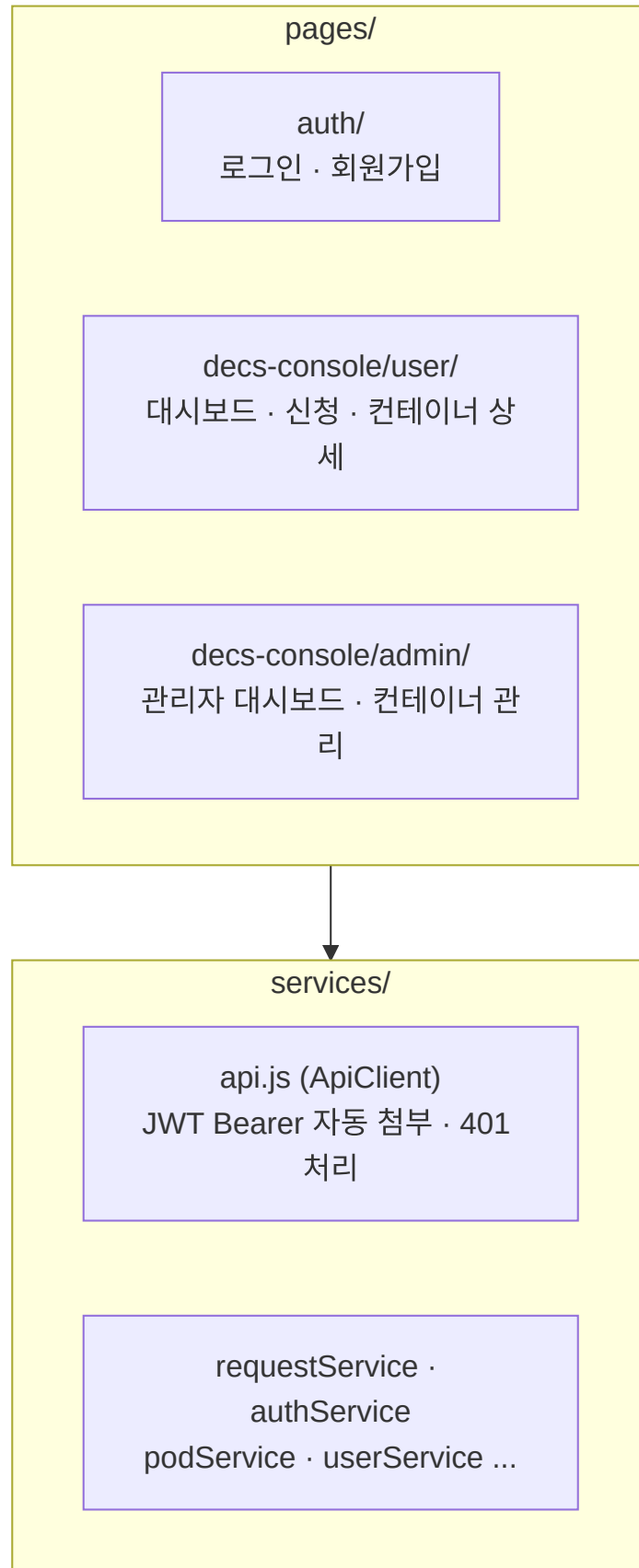
Redis List의 **RPUSH** (오른쪽 삽입)로 넣고 **LPOP** (왼쪽 꺼내기)으로 꺼내 FIFO 순서를 유지합니다. 자세한 내용은 [Redis 키 카탈로그](#) 4번을 참고합니다.



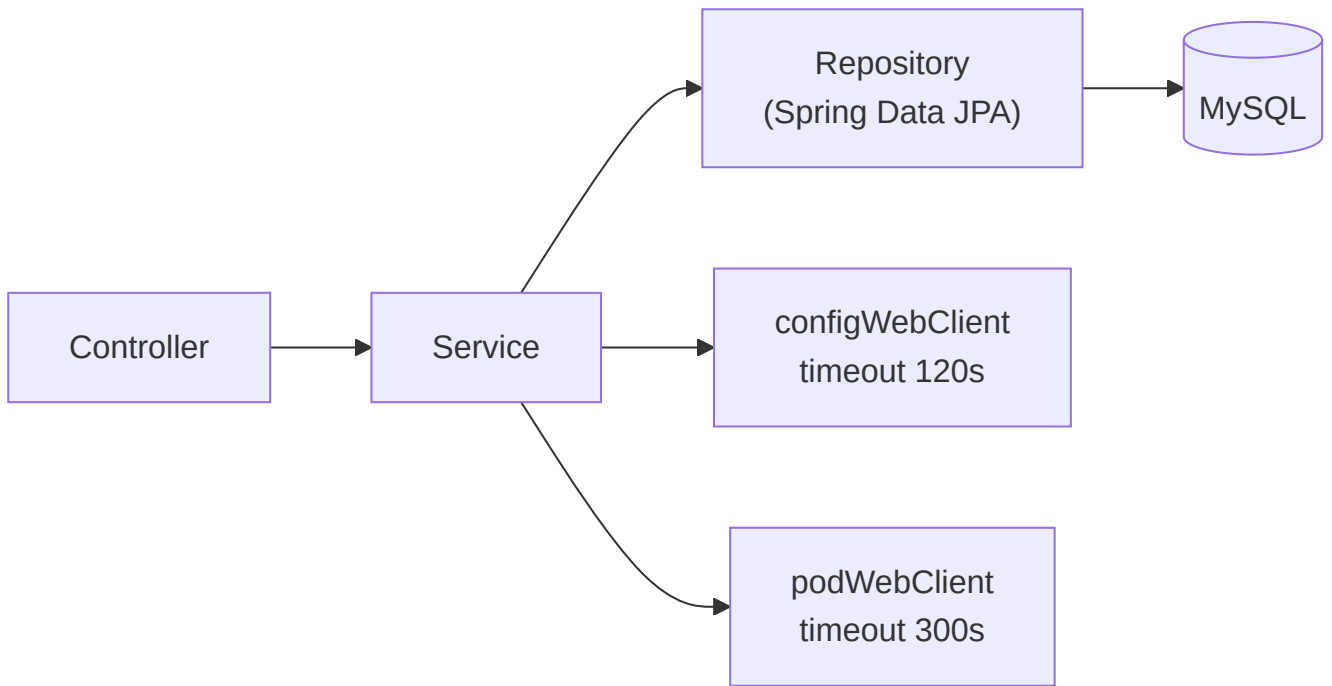
4. 내부 구조 및 통신

FE는 화면(pages)과 API 클라이언트(services)로 나뉩니다. BE는 도메인별 패키지로 구성되며 각각 Controller → Service → Repository 레이어를 따릅니다. Infra Server는 K8s 클러스터 내부 서비스로 Ubuntu 계정과 Docker 컨테이너 생성·삭제를 담당하며, BE에서만 접근합니다.

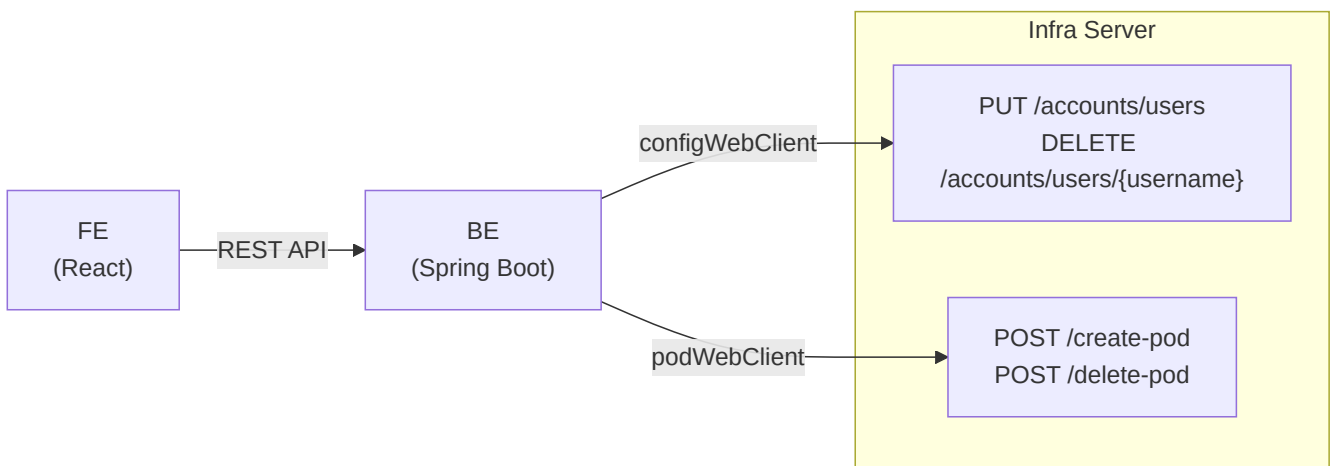
FE 내부 구조



BE 내부 구조



FE ↔ BE ↔ Infra 통신



BE 도메인 구조

domain/

requests	— 신청 접수, 승인/거절, 변경 요청 처리
users	— 사용자 관리, 인증
groups	— Ubuntu 그룹(GID) 관리
alarm	— 이메일·Slack 알림 발송
scheduler	— 만료·사용자 스케줄러, Slack 큐 Worker
pod	— Pod 정보 조회
containerImage	— 컨테이너 이미지 관리
resourceGroups	— 서버 그룹 관리
monitoring	— Prometheus 메트릭 조회
messageTemplate	— 알림 메시지 템플릿 관리

Infra Server API

K8s 클러스터 내부 서비스(`containersssh-config-service.ai lab-infra.svc.cluster.local`)로 BE에서만 접근합니다. 서비스 이름에 `containersssh` 가 포함되어 있으나, ContainerSSH 제품을 사용하는 것이 아니라 자체 개발된 Infra Server입니다. BE는 두 개의 WebClient를 사용해 통신합니다. 계정 관련 요청은 `configWebClient` (timeout 120s), Pod 관련 요청은 `podWebClient` (timeout 300s)입니다. Pod 생성에 더 긴 타임아웃을 두는 이유는 Docker 이미지 pull 시간이 포함되기 때문입니다.

엔드포인트	역할
PUT /accounts/users	Ubuntu 계정 생성, uid·gid 반환
DELETE /accounts/users/{username}	Ubuntu 계정 삭제
POST /create-pod	Docker 컨테이너 기동, SSH·Jupyter 포트 할당
POST /delete-pod	Docker 컨테이너 종료

WebClient 설정·에러 처리·보상 트랜잭션 연계 상세는 [외부 연동](#)을 참고합니다.

5. 도메인 설명

BE는 아래 도메인으로 구성됩니다. 각 도메인의 엔티티 구조, 생명주기, 비즈니스 규칙 상세는 [도메인 설명](#)을 참고합니다.

도메인	역할
users	사용자 계정 관리, 인증. USER / ADMIN 역할 구분
requests	서버 사용 신청 접수·승인·거절. ChangeRequest(변경 요청) 포함
groups	Ubuntu 그룹(GID) 관리

도메인	역할
alarm	이메일·Slack 알림 단일 진입점. 이메일 중복 방지(SETNX), Redis 큐 연계
scheduler	만료 Request 처리(08:00), 사용자 수명주기(09:00), Slack 큐 소비(1초 간격)
pod	Pod 정보 및 외부 포트(SSH·Jupyter) 관리
containerImage	컨테이너 이미지 목록 관리
resourceGroups	FARM / LAB 서버 풀 관리
monitoring	Prometheus 메트릭 조회
messageTemplate	알림 메시지 템플릿 관리. messages.properties 기본값 + DB 오버라이드

6. Redis 키 사용 현황

BE는 7개의 Redis 키 패턴을 사용합니다.

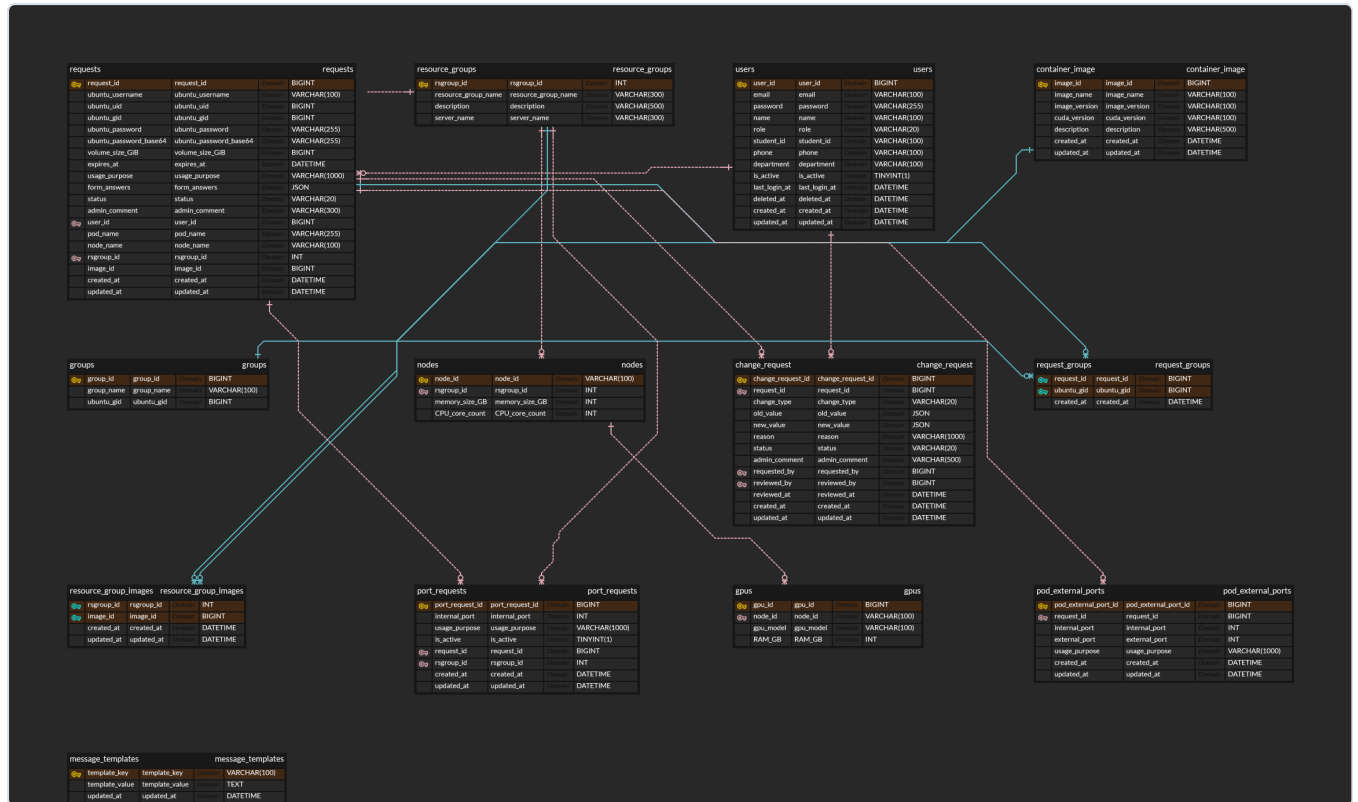
키 패턴	타입	TTL	목적
email:verify:{email}	String	5분	이메일 인증번호 임시 저장
VERIFIED:{email}	String	10분	인증 완료 상태. 이 키가 없으면 회원가입 불가
RT:{userId}	String	7일	Refresh Token. 이 키가 없으면 로그아웃 상태
email:preexpiry:{requestId}:{dayLabel}:{date}	String	25시간	만료 예고 이메일 중복 발송 방지 (SETNX)
slack:notification:queue	List	없음	Slack 알림 FIFO 큐 (1초 간격 소비)
slack:infra:notification:queue	List	없음	Infra 관련 Slack 알림 전용 큐
slack:cache:users:list	String	1시간	이메일 → Slack UserId 변환용 멤버 목록 캐시

키 패턴별 상세 설명(저장·삭제 시점, 값 형태, redis-cli 확인 명령)은 [Redis 키 카탈로그](#)를 참고합니다.

도메인 설명

코드에서 DB 조작하는 부분을 고치기 전에 "이 데이터가 왜 이런 모양으로 저장되는가"를 먼저 답해주는 문서입니다. 엔티티 하나하나가 필드를 그렇게 가진 데는 이유가 있고(예: Soft Delete 뒤에 즉시 지우지 않는 이유, 비밀번호를 두 가지 형태로 같이 저장하는 이유), 그 이유를 모르고 필드를 손대면 다른 기능이 조용히 깨집니다. 새 필드를 추가하거나 마이그레이션을 준비하기 전에는 반드시 여기서 해당 엔티티 절을 먼저 확인합니다.

ERD



ERD Cloud에서 직접 확인: [admin_be ERD](#)

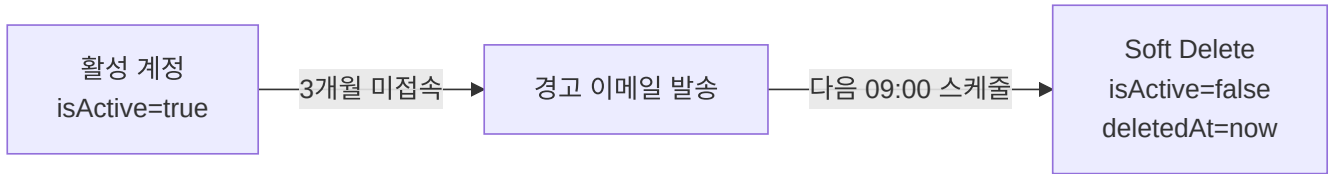
1. User

사용자 계정 정보와 생명주기를 관리합니다.

주요 필드

필드	설명
email	로그인 식별자, unique
password	BCrypt 해시
role	USER / ADMIN
isActive	계정 활성 여부. false 면 로그인 불가
lastLoginAt	마지막 로그인 시각. 스케줄러가 미접속 여부 판단에 사용
deletedAt	Soft Delete 시각. null이면 활성 계정

생명주기



Soft Delete 이후 DB row를 완전히 삭제하는 단계는 없습니다. 비활성화된 계정은 영구히 보존됩니다. 이력 보존을 위한 결정으로, `reactivate()` 로 계정을 복구할 수 있고 신청 이력·로그 추적도 계속 가능합니다.

2. Request

서버 사용 신청 한 건을 나타냅니다. 상태 전이와 보상 트랜잭션이 이 도메인의 핵심입니다.

주요 필드

필드	설명
<code>ubuntuUsername</code>	Ubuntu 계정명, unique
<code>ubuntuPassword</code> / <code>ubuntuPasswordBase64</code>	초기 비밀번호. <code>ubuntuPasswordBase64</code> 는 Infra Server API가 base64 인코딩된 평문을 요구하기 때문에 별도 보관. BCrypt 해시(<code>ubuntuPassword</code>)에서는 원문 복원이 불가능하므로 승인 시점의 base64 값을 함께 저장함
<code>ubuntuUid</code> / <code>ubuntuGid</code>	승인 후 Infra Server에서 받아 저장
<code>podName</code> / <code>nodeName</code>	승인 후 Pod 생성 결과 저장
<code>volumeSizeGiB</code>	할당 스토리지 크기
<code>expiresAt</code>	컨테이너 만료일
<code>status</code>	현재 상태 (<code>PENDING</code> / <code>PROCESSING</code> / <code>FULFILLED</code> / <code>DENIED</code> / <code>DELETED</code>)
<code>adminComment</code>	거절 사유 또는 관리자 메모
<code>formAnswers</code>	신청 양식 응답 (JSON). 양식 질문이 변경돼도 스키마 마이그레이션 없이 수용하기 위해 컬럼을 분리하지 않고 JSON으로 저장

상태 전이와 승인 흐름은 [시스템 아키텍처](#)를 참고합니다.

ChangeRequest

FULFILLED 상태의 컨테이너에 대해 사용자가 변경을 요청할 때 생성됩니다. 원본 Request와 분리된 테이블에 저장되므로, 변경 요청이 거절되거나 폐기돼도 원본 Request는 영향받지 않습니다.

changeType

타입	변경 내용
EXPIRES_AT	만료일 연장
VOLUME_SIZE	스토리지 크기 변경
RESOURCE_GROUP	서버 풀 변경
CONTAINER_IMAGE	컨테이너 이미지 변경
GROUP	Ubuntu 그룹 변경
PORT	추가 포트 요청

ChangeRequest는 PENDING → FULFILLED(승인) 또는 DENIED(거절) 상태로 전이됩니다. 만료일 연장 (EXPIRES_AT)이 승인된 경우에만 사용자에게 이메일을 발송합니다.

3. ResourceGroup / ContainerImage / Group

ResourceGroup

컨테이너를 배치할 서버 풀을 나타냅니다.

필드	설명
resourceGroupName	GPU 모델·풀 이름 (ex: 3090ti)
serverName	서버 구분 (FARM / LAB)
description	관리자용 설명

serverName 이 서버 군(FARM/LAB)을 나타내고, resourceGroupName 이 그 안의 특정 GPU 풀을 세분합니다. 관리자가 승인 시 ResourceGroup을 선택하면 Infra Server는 해당 풀 내의 노드에 Pod를 배치합니다.

ContainerImage

컨테이너 생성에 사용할 Docker 이미지 정보입니다.

필드	설명
imageName	이미지명
imageVersion	이미지 태그·버전
cudaVersion	해당 이미지가 지원하는 CUDA 버전
description	사용자에게 보여줄 설명

사용자는 서버 신청 시 목록에서 이미지를 선택합니다. CUDA 버전이 별도 필드로 관리되므로 사용자가 자신의 워크로드에 맞는 이미지를 고를 수 있습니다. `BaseTimeEntity` 를 상속하므로 `createdAt` / `updatedAt` 이 자동 관리됩니다.

Group

Ubuntu 시스템 그룹을 나타냅니다.

필드	설명
<code>groupName</code>	Ubuntu 그룹명
<code>ubuntuGid</code>	Ubuntu GID (unique)

Request와 Group은 `request_groups` 중간 테이블로 N:M 관계입니다. 하나의 신청에 여러 그룹을 지정할 수 있으며, 계정 생성 시 Infra Server의 `supplementary_groups` 필드에 이 목록이 전달됩니다. 이를 통해 컨테이너 안에서 특정 디렉터리나 리소스에 대한 접근 권한을 제어합니다.

API

엔드포인트	역할
<code>GET /api/groups</code>	전체 그룹 목록 조회. 그룹 ID·그룹명 반환
<code>POST /api/groups</code>	새 그룹 생성

그룹 삭제·수정 API는 없습니다. 잘못 만든 그룹을 정리해야 한다면 MySQL에 직접 접속합니다(운영 가이드 7절).

4. Alarm

모든 알림(이메일·Slack)은 `AlarmService` 를 단일 진입점으로 사용합니다. 직접 `JavaMailSender`나 `Slack API`를 호출하지 않고 `AlarmService`를 통해서만 발송합니다.

이메일 발송

Gmail SMTP를 통해 사용자에게 이메일을 발송합니다. 메시지 내용은 `messages.properties`의 기본값을 사용하거나, 관리자가 DB에 등록한 오버라이드 값을 우선 적용합니다(자세한 내용은 아래 `messageTemplate` 섹션 참고).

발송 대상 이벤트는 컨테이너 배정·삭제·만료일 연장·만료 예고·만료 완료뿐 아니라, 신청 거절과 변경 요청 거절도 포함합니다. 거절 이메일 발송은 try-catch로 감싸져 있어 발송 실패가 거절 처리 자체를 막지 않습니다.

만료 예고 이메일은 Redis SETNX로 중복 발송을 방지합니다. 키 형식은 `email:preexpiry:{requestId}:{dayLabel}:{date}` 이고 TTL은 25시간입니다. 스케줄러가 매일 실행되므로, 같은 날 같은 사용자에게 중복 발송되는 것을 막습니다.

Slack 발송

Slack 알림은 직접 보내지 않고 Redis List(`slack:notification:queue`)에 적재합니다. `SlackNotificationWorker`가 1초마다 LPOP으로 꺼내 Slack API를 호출합니다. Slack Webhook rate limit이 채널당 초당 1건이기 때문입니다.

5. Pod / PodExternalPort

`PodExternalPort` 는 Infra Server가 Pod 생성 시 할당한 NodePort 매핑을 저장합니다. `pod_external_ports` 테이블에 저장되며, 하나의 Request에 여러 포트가 1:N으로 매핑됩니다.

필드	타입	설명
<code>usagePurpose</code>	<code>VARCHAR(1000)</code>	포트 용도 (<code>ssh</code> , <code>jupyter</code> 등)
<code>internalPort</code>	<code>INT</code> (1-65535)	컨테이너 내부 포트
<code>externalPort</code>	<code>INT</code> (1-65535)	외부에서 접속하는 Kubernetes NodePort 번호

`BaseTimeEntity` 를 상속하므로 `createdAt` / `updatedAt` 이 자동 관리됩니다.

흐름 요약

1. 관리자가 신청 승인 → `POST /create-pod` 호출
2. Infra Server가 NodePort를 자동 할당하고 `ports` 배열로 응답
3. BE가 배열을 순회하며 `PodExternalPort` row를 `request_id` 에 묶어 저장
4. 승인 완료 이메일에 포트 목록이 포함되어 사용자에게 발송

사용자는 이 포트를 통해 컨테이너에 SSH(`ssh -p {externalPort} ubuntu@서버IP`)나 Jupyter(`http://서버IP:{externalPort}`)로 접속합니다.

6. Node / Gpu

물리 서버 노드와 GPU 사양을 DB에서 관리합니다. Prometheus가 실시간 메트릭을 수집하는 것과 달리, Node/Gpu 테이블은 정적인 하드웨어 목록(어떤 서버에 GPU가 몇 개, 어떤 모델인지)을 저장합니다. 관리자 대시보드에서 서버 목록을 표시하거나, 신청 폼에서 ResourceGroup 설명을 구성할 때 이 정보를 사용합니다.

Node

필드	설명
<code>nodeId</code>	노드 식별자 (ex: <code>farm1</code> , <code>lab-dgx</code>) — 문자열 PK, Infra Server의 노드명과 일치
<code>rsgroup_id</code>	소속 ResourceGroup (FARM / LAB)
<code>memorySizeGB</code>	시스템 RAM (GB)
<code>cpuCoreCount</code>	CPU 코어 수

Gpu

필드	설명
gpuModel	GPU 모델명 (ex: RTX4050 , A3000)
ramGb	GPU VRAM (GB)
node_id	소속 노드 FK

하나의 노드에 여러 GPU가 장착될 수 있어 Node → Gpu는 1:N입니다.

7. PortRequests

사용자가 서버 신청 시 SSH·Jupyter 외에 추가 포트를 요청하는 경우 `port_requests` 테이블에 저장됩니다. 기본 제공 포트(`PodExternalPort`)와 달리, 사용자가 명시적으로 요청한 포트를 별도로 추적합니다.

필드	설명
internalPort	요청하는 컨테이너 내부 포트
usagePurpose	포트 용도 (ex: tensorboard , custom-api)
isActive	실제 Infra에 할당 완료 여부. 신청 시 <code>false</code> , 관리자 승인 후 포트가 열리면 <code>true</code>
request_id	연관 Request FK
rsgroup_id	어느 서버 풀에 요청하는지

`isActive=false` 는 "요청은 됐지만 아직 Infra에 포트가 열리지 않은 상태"를 의미합니다. `activate()` 가 호출되면 `isActive=true` 로 전환됩니다.

8. Monitoring / Dashboard

Monitoring

`GET /api/monitoring/metrics` — FE 대시보드가 주기적으로 폴링합니다. BE가 Prometheus HTTP API에 PromQL 쿼리를 날려 결과를 가공해 반환합니다. 인증 없이 접근 가능한 화이트리스트 경로(`/api/monitoring/**`)입니다.

지표	PromQL
서버별 평균 GPU 사용률 (%)	<code>avg by(Hostname)(DCGM_FI_DEV_GPU_UTIL)</code>
서버별 GPU 개수	<code>count by(Hostname)(DCGM_FI_DEV_GPU_UTIL)</code>
클러스터별 활성 컨테이너 수	<code>sum by(cluster)(cluster_monitor_container_running)</code>

Prometheus가 일시적으로 내려가도 API는 200을 반환합니다(빈 값 응답). 각 쿼리에 5초 타임아웃이 적용됩니다. Prometheus 연동 상세(WebClient 설정, 응답 구조 등)는 [외부 연동](#) 섹션 4를 참고합니다.

Dashboard

`GET /api/admin/dashboard` — 관리자 대시보드용 집계 데이터를 단일 API로 내려줍니다.

집계 항목	내용
신청 현황	상태별 Request 수 (<code>PENDING</code> / <code>FULFILLED</code> / <code>DENIED</code> 등)
사용자 현황	전체 사용자 수, 활성 사용자 수
서버별 사용량	ResourceGroup별 FULFILLED Request 수

9. MessageTemplate

알림 메시지를 재배포 없이 수정할 수 있도록 "DB 우선, properties 폴백" 구조로 관리합니다.

`messages.properties`에 모든 메시지 키의 기본값이 있습니다. JAR에 포함되므로 수정하려면 재배포가 필요합니다. 관리자 화면에서 특정 키의 값을 수정하면 `message_templates` 테이블에 row가 생성됩니다. `DbMessageSource`는 메시지를 읽을 때 DB를 먼저 확인하고, 없으면 `messages.properties`로 폴백합니다.

메시지 조회 순서:

1. `message_templates` 테이블에 해당 key row가 있으면 → DB 값 사용
2. 없으면 → `messages.properties` 기본값 사용

관리자 API를 통해 특정 키를 "초기화(reset)"하면 DB row가 삭제되어 기본값으로 돌아갑니다. 코드를 건드리지 않고 이메일/Slack 메시지 내용을 조정할 때 유용합니다.

코드 컨벤션

사람이 바뀌어도 코드는 한 사람이 짰 것처럼 보여야 유지보수가 됩니다. 그 일관성을 지키기 위한 규칙과, 그 규칙이 왜 그렇게 정해졌는지의 근거를 함께 정리했습니다. Entity에 `@Data`를 쓰지 않는 이유, Setter 대신 의도가 드러나는 메서드를 쓰는 이유가 그런 예입니다. 규칙만 외우면 애매한 새 상황에서 판단이 서지 않으므로, 이유까지 같이 알아둡니다. 코드에서 "왜 이렇게 짜여 있지?" 싶은 부분이 있다면 먼저 여기서 답을 찾습니다.

1. 패키지 구조

도메인 중심으로 패키지를 구성합니다. 각 도메인 안에는 아래 하위 패키지가 들어갑니다.

```

domain/
├── {도메인}/
│   ├── controller/
│   │   ├── docs/           # Swagger 인터페이스
│   │   ├── service/        # CommandService / QueryService 분리
│   │   ├── entity/
│   │   ├── repository/
│   │   ├── dto/
│   │   │   ├── request/
│   │   │   └── response/

```

전체 도메인 목록: `requests`, `users`, `groups`, `alarm`, `scheduler`, `pod`, `containerImage`, `resourceGroups`, `monitoring`, `messageTemplate`, `dashboard`

공통 기능은 `global` 패키지 아래 `auth`, `config`, `common`, `util` 로 분리합니다.

2. 클래스/파일 네이밍

사용자용과 관리자용 클래스를 분리합니다. 관리자 전용에는 반드시 `Admin` prefix를 붙여서, 파일명만 봐도 어느 쪽 기능인지 구분할 수 있게 합니다.

Controller는 역할이 명확하면 그 역할을 이름에 드러냅니다. 예를 들어 신청 변경 요청만 전담하는 컨트롤러는 `AdminRequestController` 가 아니라 `AdminRequestChangeController` 로 씁니다.

Service는 읽기(`QueryService`)와 쓰기(`CommandService`)를 분리합니다. 특정 역할에만 집중하는 서비스는 그 역할을 이름에 포함합니다(`UserLoginService`, `RequestExpiryService`, `UbuntuAccountService`).

DTO는 목적과 방향을 이름에 담습니다. Request DTO는 어떤 동작에 쓰이는지(`SaveRequestRequestDTO`, `ApproveRequestDTO`), Response DTO는 무엇을 반환하는지(`UserResponseDTO`, `MyInfoResponseDTO`) 이름에서 바로 알 수 있어야 합니다. suffix는 `DTO` 로 통일합니다.

유형	예시
Controller (사용자)	<code>RequestController</code> , <code>UserController</code> , <code>GroupController</code>
Controller (관리자)	<code>AdminRequestController</code> , <code>AdminUserController</code> , <code>AdminRequestChangeController</code>
Service (읽기)	<code>RequestQueryService</code> , <code>AdminRequestQueryService</code>
Service (쓰기)	<code>RequestCommandService</code> , <code>AdminRequestCommandService</code>
Service (단일 역할)	<code>UserLoginService</code> , <code>RequestExpiryService</code> , <code>UbuntuAccountService</code>
Repository	<code>RequestRepository</code> , <code>ChangeRequestRepository</code>
Entity	<code>Request</code> , <code>User</code> , <code>Group</code> , <code>ContainerImage</code> , <code>ChangeRequest</code>

유형	예시
Request DTO	SaveRequestRequestDTO , ApproveRequestDTO , CreateGroupRequestDTO
Response DTO	UserResponseDTO , SaveRequestResponseDTO , MyInfoResponseDTO

3. 메서드 네이밍

Service 메서드는 동사 접두사로 역할을 구분합니다. 조회는 `get*`, 생성은 `create*`, 수정은 `update*` 를 기본으로 쓰고, 승인/거절처럼 도메인 동사가 명확한 경우엔 그대로 씁니다.

동작	접두사	예시
조회	<code>get*</code>	<code>getMyInfo</code> , <code>getRequestsByUserId</code> , <code>getAllActiveContainers</code>
생성	<code>create*</code>	<code>createRequest</code> , <code>createGroup</code> , <code>createModificationRequest</code>
수정	<code>update*</code>	<code>updatePassword</code> , <code>updatePhone</code>
승인/거절	동사 그대로	<code>approveRequest</code> , <code>rejectRequest</code> , <code>approveModification</code>
상태 전환	동사 그대로	<code>markAsProcessing</code> , <code>revertToPending</code> , <code>deleteAfterCleanup</code>

엔티티 내부 메서드도 같은 방식입니다. `Request` 엔티티는 `approve()`, `reject()`, `markAsProcessing()`, `revertToPending()`, `assignPodInfo()` 처럼 동작을 직접 드러내는 이름을 씁니다. 외부에서 setter로 상태를 바꾸는 것을 막고, 엔티티 스스로 상태를 관리하게 하기 위해서입니다.

Repository 메서드는 Spring Data 네이밍 컨벤션을 따릅니다.

```

findByEmail(String email)
findAllByUser_UserId(Long userId)
existsByUbuntuUsernameAndStatusIn(String username, List<Status> statuses)

```

4. BaseTimeEntity

모든 주요 엔티티는 `BaseTimeEntity` 를 상속합니다. 엔티티마다 `createdAt`, `updatedAt` 컬럼을 중복 정의하지 않고 한 곳에서 관리하기 위해서입니다. `@EntityListeners(AuditingEntityListener.class)` 가 붙어 있어 생성/수정 시각이 자동으로 기록됩니다.

- 위치: `DGU_AI_LAB.admin_be.global.common.BaseTimeEntity`

```
public class Request extends BaseTimeEntity {  
    // createdAt, updatedAt 자동 관리  
}
```

5. Entity 작성 규칙

필수/권장 어노테이션

- `@Entity`, `@Table(name = "...")`
- `@Id`, `@GeneratedValue(strategy = GenerationType.IDENTITY)`
- `@Column(nullable = false, ...)` — 제약 조건은 명시
- `@Enumerated(EnumType.STRING)` — ORDINAL로 저장하면 enum 순서가 바뀔 때 데이터가 틀어지므로 STRING 사용
- 연관관계는 기본 `FetchType.LAZY` — EAGER는 불필요한 쿼리를 유발합니다

Lombok 조합

```
@Getter  
@NoArgsConstructor(access = AccessLevel.PROTECTED)  
@Entity  
public class User extends BaseTimeEntity { }
```

`@NoArgsConstructor(access = AccessLevel.PROTECTED)` 는 JPA가 요구하는 기본 생성자를 만들면서, 외부에서 무분별하게 빈 객체를 생성하는 것을 막기 위해 `PROTECTED` 로 제한합니다.

`@Data` 는 Entity에서 사용 금지입니다. 무분별한 Setter 생성, `toString()` 의 연관 엔티티 LAZY 로딩 트리거, JPA 프록시 충돌 문제가 생깁니다.

비즈니스 메서드

엔티티가 상태를 직접 변경하는 메서드를 가집니다. 외부에서 setter로 필드를 직접 바꾸는 대신, 엔티티에 의도를 드러내는 메서드를 두어 변경 경로를 하나로 통일합니다.

```

public void approve(ContainerImage image, ResourceGroup resourceGroup, ...) {
    this.containerImage = image;
    this.status = Status.FULFILLED;
    this.approvedAt = LocalDateTime.now();
}

public void markAsProcessing() {
    this.status = Status.PROCESSING;
}

```

6. DTO 작성 규칙

DTO는 불변(immutable)으로 작성합니다. 값이 바뀌면 버그 추적이 어렵기 때문입니다. Java `record` 를 기본으로 사용합니다.

Request DTO에는 Validation 어노테이션을 달아두고, Controller에서 `@Valid` 로 검증을 트리거합니다. Response DTO에는 `fromEntity()` static 메서드를 두어 엔티티 → DTO 변환을 한 곳에서 관리합니다. Swagger 문서화를 위해 `@Schema` 도 포함합니다.

```

@Builder
public record SaveRequestRequestDTO(
    @NotNull(message = "Resource group ID cannot be null")
    Integer resourceGroupId,

    @NotBlank(message = "Ubuntu username cannot be blank")
    @Size(min = 3, max = 50)
    String ubuntuUsername,

    @Future(message = "Expires date must be in the future")
    LocalDateTime expiresAt
) {
    public Request toEntity(User user, ...) { ... }
}

```

`@Data` 는 DTO에서도 사용하지 않습니다.

7. 각 클래스 어노테이션 정리

Controller

Controller는 HTTP 요청 수신, 인증 정보 추출, Service 호출, 응답 반환만 담당합니다. 비즈니스 로직은 Service에서 처리합니다. Swagger 문서는 컨트롤러 클래스에 직접 달지 않고 `controller/docs/` 아래 인터페이스를 만들어 분리합니다.

- `@RestController` — JSON API 응답 표준
- `@RequestMapping` — 클래스 레벨에서 공통 경로 prefix 지정 (예: `/api/requests`)
- `@Valid` — 요청 DTO 유효성 검증 트리거

Service

- `@Transactional` — 쓰기 작업에 적용. 중간에 실패하면 전체 롤백됩니다.
- `@Transactional(readOnly = true)` — 조회 작업에 적용. flush와 dirty checking을 건너뛰어 성능이 좋습니다.
- `@Slf4j` — `System.out.println` 대신 SLF4J 로깅 사용

8. CQS (Command Query Separation)

조회와 변경을 서비스 레벨에서 분리합니다. 한 서비스가 읽기와 쓰기를 모두 담당하면 트랜잭션 옵션을 일관되게 적용하기 어렵고, 책임이 불분명해집니다.

- **QueryService:** 조회만 담당. `@Transactional(readOnly = true)` 적용
- **CommandService:** 변경만 담당. `@Transactional` 적용

```
@Service
@Transactional(readOnly = true)
public class RequestQueryService {
    public List<SaveRequestResponseDTO> getRequestsByUserId(Long userId) { ... }
}
```

Query 코드에서 엔티티 값을 변경하지 않습니다.

9. 정적 팩토리 메서드

`new` 생성자 대신 `static` 팩토리 메서드를 사용합니다. 생성 의도를 이름으로 드러낼 수 있고, 생성 규칙을 한 곳에서 관리할 수 있습니다.

```
// Response DTO에서 엔티티 변환
public static UserResponseDTO fromEntity(User user) {
    return new UserResponseDTO(user.getId(), user.getName(), user.isActive());
}
```

10. 예외 처리 및 ErrorCode

에러는 아래 흐름으로 처리됩니다.

1. Service에서 조건 미충족 시 `BusinessException` 발생
2. `GlobalExceptionHandler` 가 예외를 가로챈
3. `ErrorCode` 에 정의된 HTTP 상태 코드와 메시지로 클라이언트에 응답

예외 계층

```
BusinessException
├── EntityNotFoundException
└── UnauthorizedException
```

사용 방법

예외는 Service 레이어에서만 던집니다. `ErrorCode` 를 직접 넘겨서 응답 메시지와 상태 코드를 한 곳에서 관리합니다.

```
// 조회 실패
User user = userRepository.findById(userId)
    .orElseThrow(() -> new BusinessException(ErrorCode.USER_NOT_FOUND));

// 비즈니스 규칙 위반
if (!passwordEncoder.matches(request.currentPassword(), user.getPassword())) {
    throw new BusinessException(ErrorCode.INVALID_PASSWORD);
}
```

ErrorCode 정의

위치: `DGU_AI_LAB.admin_be.error.ErrorCode`

새 에러 케이스가 생기면 enum에 추가합니다. 유사한 상태 코드끼리 묶어서 작성하고, 메시지는 FE에 바로 노출될 수 있으므로 명확한 한글로 작성합니다. 현재 정의된 59개 케이스 전체 목록은 [에러 코드 카탈로그](#)를 참고합니다.

```

@Getter
@RequiredArgsConstructor(access = AccessLevel.PRIVATE)
public enum ErrorCode {
    // 400 Bad Request
    INVALID_INPUT_VALUE(HttpStatus.BAD_REQUEST, "입력값이 올바르지 않습니다."),

    // 401 Unauthorized
    EXPIRED_ACCESS_TOKEN(HttpStatus.UNAUTHORIZED, "엑세스 토큰이 만료되었습니다."),

    // 404 Not Found
    RESOURCE_NOT_FOUND(HttpStatus.NOT_FOUND, "해당 리소스를 찾을 수 없습니다."),

    // 500 Internal Server Error
    INTERNAL_SERVER_ERROR(HttpStatus.INTERNAL_SERVER_ERROR, "서버 내부 오류입니다.");

    private final HttpStatus httpStatus;
    private final String message;
}

```

11. API 응답 포맷

성공 응답은 `SuccessResponse`, 실패 응답은 `ErrorResponse` 를 사용합니다. 응답 포맷을 통일해서 FE가 일관된 방식으로 처리할 수 있게 합니다.

```

// 성공
{ "status": 200, "message": "요청이 성공했습니다.", "data": { ... } }

// 실패
{ "status": 404, "message": "사용자를 찾을 수 없습니다." }

```

```

return SuccessResponse.ok(body);
return SuccessResponse.created(body);

```

12. 로깅 규칙

레벨을 구분해서 찍어야 운영 중 로그 필터링이 수월합니다. 레벨별 기준은 아래와 같습니다.

레벨	사용 상황
INFO	주요 비즈니스 이벤트 (요청 생성, 승인, 삭제 등)
WARN	비즈니스 규칙 위반, 중복 감지 등
DEBUG	상세 디버깅 정보
ERROR	시스템 오류, 외부 API 실패

메서드명을 **[대괄호]** 로 표시해 어디서 찍힌 로그인지 빠르게 파악할 수 있게 합니다.

```
log.info("[createRequest] 요청 생성: userId={}, resourceGroupId=", userId, dto.resourceGroupId());
log.warn("[createRequest] 중복 username 감지: {}", dto.ubuntuUsername());
log.error("[approveRequest] 인프라 API 호출 실패", e);
```

13. 쿼리 최적화

연관 엔티티를 나중에 따로 조회하면 N+1 문제가 생깁니다. 한 번의 쿼리로 필요한 연관 엔티티를 모두 가져오려면 FETCH JOIN을 사용합니다.

```
@Query("SELECT DISTINCT r FROM Request r " +
        "JOIN FETCH r.user " +
        "LEFT JOIN FETCH r.resourceGroup " +
        "WHERE r.status = :status")
List<Request> findAllByStatusWithAssociations(@Param("status") Status status);
```

14. PR 및 코드 리뷰

기능 개발, 버그 수정 가릴 것 없이 반드시 PR을 남기고 팀원에게 코드 리뷰를 받은 뒤 머지합니다. 리뷰 없이 main/develop에 직접 푸시하지 않습니다.

PR 작성 시:

- 변경 사항과 이유를 설명합니다.
- `application.yml` 을 수정했다면 PR 본문에 명시하고 Notion 페이지도 함께 업데이트합니다.
- 리뷰어는 로직뿐 아니라 컨벤션 준수 여부도 확인합니다.
- **커밋 전에 반드시 테스트를 실행하고, 모든 테스트가 통과하는 상태에서만 커밋합니다.** 테스트가 깨진 코드를 커밋하지 않습니다.

15. 테스트 작성

로컬에서 테스트 실행: 별도 DB 설정 없이 바로 실행됩니다. `src/test/resources/application.yml` 이 `test` 프로파일과 H2 인메모리 DB(`jdbc:h2:mem:testdb`)를 자동으로 사용하기 때문입니다.

```
./gradlew test
```

CI(`deploy.yml`)는 배포 속도를 위해 `./gradlew build -x test`로 테스트를 건너뜁니다. 즉 테스트 통과 여부는 CI가 아니라 PR 리뷰와 로컬 실행에 전적으로 의존합니다. 커밋 전 테스트 실행이 규칙(14절)인 이유이기도 합니다.

테스트 케이스는 가능한 한 촘촘하게 작성합니다. 주요 흐름(happy path)뿐 아니라 엣지 케이스, 실패 케이스, 보상 트랜잭션 동작까지 커버합니다.

테스트는 세 가지 레이어로 구분합니다.

- **Service 단위 테스트:** 비즈니스 로직 검증. 외부 의존성은 Mockito로 대체합니다.
- **Controller 통합 테스트:** HTTP 요청/응답 검증. MockMvc로 실제 요청을 재현합니다.
- **Repository 테스트:** 커스텀 쿼리 검증. H2 인메모리 DB를 사용합니다.

테스트 메서드명

한국어로 작성하며 `@DisplayName`으로 의도를 명확히 설명합니다. 메서드명과 `DisplayName`을 조합해 어떤 상황에서 어떤 결과를 기대하는지 바로 알 수 있어야 합니다.

```
@Test
@DisplayName("이메일 인증이 완료되고 중복이 없으면 회원가입에 성공한다")
void register_success() { ... }

@Test
@DisplayName("이미 가입된 이메일로 회원가입하면 BusinessException을 던진다")
void register_throwsException_whenEmailAlreadyExists() { ... }
```

연관된 테스트 케이스는 `@Nested` + `@DisplayName`으로 묶어 가독성을 높입니다.

```

@Nested
@DisplayName("login (로그인)")
class Login {

    @Test
    @DisplayName("올바른 이메일과 비밀번호로 로그인하면 토큰을 반환한다")
    void login_success() { ... }

    @Test
    @DisplayName("비밀번호가 틀리면 UnauthorizedException을 던진다")
    void login_throwsException_whenPasswordWrong() { ... }
}

```

Service 단위 테스트

`@ExtendWith(MockitoExtension.class)` 를 사용합니다. 테스트 대상(`@InjectMocks`)과 의존성(`@Mock`)을 선언하고, `@BeforeEach` `setUp()` 에서 공통 픽스처를 구성합니다.

`@Value` 로 주입받는 필드는 Mockito가 처리하지 못하므로 `ReflectionTestUtils.setField()` 로 직접 설정합니다.

```

@ExtendWith(MockitoExtension.class)
class UserLoginServiceTest {

    @InjectMocks
    private UserLoginService userLoginService;

    @Mock private UserRepository userRepository;
    @Mock private PasswordEncoder passwordEncoder;
    @Mock private JwtProvider jwtProvider;
    @Mock private RedisTemplate<String, String> redisTemplate;

    @BeforeEach
    void setUp() {
        // @Value 필드는 Mockito가 주입하지 않으므로 직접 설정
        ReflectionTestUtils.setField(userLoginService, "REFRESH_TOKEN_EXPIRE_TIME", 604800000L);
    }
}

```

예외 발생 검증은 `assertThatThrownBy()` 로 씁니다.

```

assertThatThrownBy(() -> userLoginService.login(dto))
    .isInstanceOf(UnauthorizedException.class);

```

특정 ErrorCode까지 검증하려면 `extracting()` 을 사용합니다.

```
assertThatThrownBy(() -> service.approveRequest(dto))
    .isInstanceOf(BusinessException.class)
    .extracting(e -> ((BusinessException) e).getErrorCode())
    .isEqualTo(ErrorCode.INVALID_REQUEST_STATUS);
```

메서드 호출 횟수와 인자를 검증할 때는 `verify()` 와 `ArgumentCaptor` 를 활용합니다.

```
// 정확히 1회 호출 확인
verify(passwordEncoder, times(1)).matches("password123", "encodedPassword");

// 호출되지 않아야 함 확인
verify(podService, never()).deletePod(any());

// 실제 저장된 값 캡처 후 검증
ArgumentCaptor<PodExternalPort> captor = ArgumentCaptor.forClass(PodExternalPort.class);
verify(podExternalPortRepository, times(2)).save(captor.capture());
assertThat(captor.getAllValues().get(0).getUsagePurpose()).isEqualTo("ssh");
```

의존성이 많아 `@InjectMocks` 가 불안정한 경우, `@BeforeEach` 에서 생성자로 직접 주입합니다.

```
@BeforeEach
void setUp() {
    // @RequiredArgsConstructor 생성자 필드 선언 순서대로 주입
    service = new AdminRequestCommandService(
        alarmService, requestRepository, userRepository, ...
    );
}
```

Controller 통합 테스트

`@WebMvcTest` 로 특정 컨트롤러만 로드합니다. Spring Security는 컨트롤러 테스트 범위를 벗어나므로 `excludeAutoConfiguration` 으로 제외합니다. 모든 컨트롤러 테스트는 `WebMvcTestSupport` 를 상속해 공통 Mock 설정을 받습니다.

```

@WebMvcTest(
    value = AuthController.class,
    excludeAutoConfiguration = {SecurityAutoConfiguration.class}
)
class AuthControllerTest extends WebMvcTestSupport {

    @Autowired MockMvc mockMvc;
    @Autowired ObjectMapper objectMapper;

    @MockitoBean
    private UserLoginService userLoginService;
}

```

`WebMvcTestSupport`는 `SecurityConfig`가 필요로 하는 공통 빈(`JwtProvider`, `CustomUserDetailsService`, `RedisTemplate`, `JpaMetamodelMappingContext`)을 Mock으로 등록한 추상 클래스입니다. 새 컨트롤러 테스트를 작성할 때는 항상 이를 상속합니다.

HTTP 요청은 `mockMvc.perform()`으로, 응답은 `andExpect()`로 검증합니다.

```

// 성공 케이스: 201 Created
mockMvc.perform(post("/api/auth/register")
    .contentType(MediaType.APPLICATION_JSON)
    .content(objectMapper.writeValueAsString(dto)))
    .andExpect(status().isCreated());

// 실패 케이스: 예외 → HTTP 상태 코드 매핑 확인
doThrow(new BusinessException(ErrorCode.USER_ALREADY_EXISTS))
    .when(userLoginService).register(any());

mockMvc.perform(post("/api/auth/register")
    .contentType(MediaType.APPLICATION_JSON)
    .content(objectMapper.writeValueAsString(dto)))
    .andExpect(status().isConflict());

// 응답 본문 JSON 필드 검증
mockMvc.perform(post("/api/auth/login")...)
    .andExpect(status().isOk())
    .andExpect(jsonPath("$.accessToken").value("accessToken"));

```

Repository 테스트

`@DataJpaTest` + `@ActiveProfiles("test")` 를 사용합니다. H2 인메모리 DB로 실제 쿼리를 실행해 검증하므로, JPQL이나 `@Query` 의 적합성을 확인할 수 있습니다.

```
@DataJpaTest
@ActiveProfiles("test")
class UserRepositoryTest {

    @Autowired
    private UserRepository userRepository;

    @BeforeEach
    void setUp() {
        userRepository.save(User.builder()
            .email("active@dgu.ac.kr")
            .password("encoded1")
            .name("홍길동")
            ...
            .build());
    }
}
```

엔티티의 private 필드(예: `lastLoginAt`, `deletedAt`)를 테스트용으로 설정해야 할 때는 `ReflectionTestUtils.setField()` 를 사용합니다. 변경 후에는 `flush()` 를 호출해 DB에 즉시 반영합니다.

```
ReflectionTestUtils.setField(user1, "lastLoginAt", LocalDateTime.now().minusMonths(4));
userRepository.save(user1);
userRepository.flush();

List<User> result = userRepository.findInactiveUsers(LocalDateTime.now().minusMonths(3));
assertThat(result).extracting(User::getEmail).contains("active@dgu.ac.kr");
```

16. 사용자 권한 관리

별도로 유저 `Role` 을 변경하는 API를 제공하지 않습니다. 필요한 경우 직접 서버실을 방문하여 데이터베이스에서 변경합니다.

핵심 설계 패턴

같은 문제를 팀원마다 다른 방식으로 풀면 코드베이스가 제각각이 됩니다. BE에는 "DB 커밋 전에 알림이 나가면 안 된다", "배치가 하루에 두 번 돌아도 중복 발송되면 안 된다"처럼 여러 도메인에서 반복해서 마주치는 문제가 있고, 그때마다 이미 검증된 해결 방식이 정해져 있습니다. 새 기능을 만들다가 낯익은 문제를 만나면, 새로 고민하기 전에 여기에 이미 답이 있는지부터 확인합니다.

1. AFTER_COMMIT — DB 커밋 후 알림 발송

문제

서비스 메서드가 `@Transactional` 안에서 이메일·Slack 알림을 직접 발송하면, DB 커밋 전에 알림이 나갑니다. 이후 트랜잭션이 롤백되면 "DB엔 저장 안 됐지만 이메일은 발송된" 상태가 됩니다.

해결

Spring의 `@TransactionalEventListener(phase = AFTER_COMMIT)` 를 사용합니다. DB 커밋이 확정된 이후에만 이벤트 리스너가 실행되므로, 롤백 시 이벤트가 자동으로 폐기됩니다.

```
// 1. 이벤트 발행 (트랜잭션 안에서)
eventPublisher.publishEvent(new RequestExpiredEvent(userName, userEmail, ...));

// 2. 리스너 (커밋 후 실행)
@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
public void onRequestExpired(RequestExpiredEvent event) {
    alarmService.sendEmail(event.userEmail(), ...);
}
```

적용 범위

만료 Request 삭제, 관리자 수동 삭제, 승인 완료 등 DB 변경과 알림이 짝을 이루는 모든 흐름에 적용합니다.

2. Self-Injection — 같은 클래스 내 @Transactional 호출

문제

Spring `@Transactional` 은 AOP 프록시로 동작합니다. 같은 클래스 내에서 `this.method()` 로 호출하면 프록시를 우회해 트랜잭션이 적용되지 않습니다.

```
// 잘못된 예 — @Transactional이 적용되지 않음
public void runScheduler() {
    this.deleteExpiredRequest(id); // 프록시 우회
}
```

해결

@Transactional 메서드를 별도 클래스로 분리하고, 해당 Bean을 주입받아 호출합니다. 분리된 Bean은 Spring이 프록시로 감싸기 때문에 트랜잭션이 정상 적용됩니다.

```
// RequestSchedulerService — 루프 제어만 담당
@Service
@RequiredArgsConstructor
public class RequestSchedulerService {

    private final RequestExpiryService requestExpiryService; // 별도 클래스 주입

    public void runScheduler() {
        requestExpiryService.deleteExpiredRequest(id); // 프록시를 통해 호출 → @Transactional 적용
    }
}

// RequestExpiryService — 트랜잭션 경계 담당
@Service
@RequiredArgsConstructor
public class RequestExpiryService {

    @Transactional
    public void deleteExpiredRequest(Long id) { ... }
}
```

Self-injection(`private final RequestSchedulerService self`)도 같은 문제를 해결하는 패턴이지만, 순환 의존 가능성을 피하기 위해 이 프로젝트에서는 별도 클래스 분리 방식을 사용합니다.

적용 범위

스케줄러처럼 루프 안에서 건별 독립 트랜잭션이 필요한 경우에 사용합니다. 한 건이 실패해도 나머지 건이 롤백되지 않습니다.

3. SETNX 중복 방지

문제

스케줄러는 매일 같은 시각에 실행됩니다. 같은 사용자에게 만료 예고 이메일이 하루에 두 번 발송되면 안 됩니다.

해결

Redis **SETNX** (SET if Not eXists)로 "오늘 이미 발송했는지"를 체크합니다. 키가 없으면 발송하고 키를 생성합니다. 키가 이미 있으면 건너뛵니다. TTL을 25시간으로 설정해 하루가 지나면 자동으로 만료됩니다.

```
String key = "email:preexpiry:" + requestId + ":" + dayLabel + ":" + date;
Boolean set = redisTemplate.opsForValue().setIfAbsent(key, "sent", Duration.ofHours(25));
if (Boolean.FALSE.equals(set)) {
    return; // 이미 발송됨 — 건너뛸
}
// 발송 로직
```

Fail-open

Redis 장애 시 **setIfAbsent()** 가 예외를 던집니다. 이때 **false** 를 반환해 이메일 발송을 허용합니다(fail-open). Redis 장애 때문에 사용자가 알림을 못 받는 것보다, 중복 발송이 낫다는 판단입니다.

적용 범위

동일 이벤트의 중복 처리를 막을 때 이 패턴을 사용합니다. 키 형식은 **{도메인}:{식별자}:{날짜}** 로 구성하고, TTL은 실행 주기보다 정확히 여유를 두고 설정합니다. 예를 들어 매일 08:00에 도는 스케줄러(24시간 주기)라면 TTL은 25시간으로, 실행 시각이 다소 밀려도 흡수할 수 있게 1시간을 여유로 둡니다.

4. 3단 보상 트랜잭션

문제

관리자 승인 흐름은 Ubuntu 계정 생성 → Pod 생성 → DB 저장 세 단계입니다. 중간에 실패하면 이미 완료된 단계를 되돌려야 합니다.

해결

각 단계 실패 시 역순으로 정리합니다.

```
Ubuntu 계정 생성 → 성공 → Pod 생성 → 실패 → Ubuntu 계정 삭제 → Request PENDING 복구
→ 실패 → BusinessException 전파 (Pod 생성 안 됐으니 정리 불필요)
```

```

try {
    ubuntuService.createAccount(username);    // 1단계
} catch (Exception e) {
    throw new BusinessException(UBUNTU_CREATION_FAILED); // 정리 불필요
}

try {
    podService.createPod(username);           // 2단계
} catch (Exception e) {
    ubuntuService.deleteAccount(username);    // 1단계 보상
    request.revertToPending();
    throw new BusinessException(POD_CREATION_FAILED);
}

// 3단계: DB 저장 (@Transactional 롤백이 자동 처리)

```

주의

보상 작업(`deleteUbuntuAccount`)이 실패해도 원래 예외(`POD_CREATION_FAILED`)를 그대로 전파합니다. 보상 실패는 로그로만 남기고, Slack으로 관리자에게 알립니다.

5. Redis 큐를 통한 비동기 처리

문제

Slack Webhook API는 채널당 초당 1건의 rate limit이 있습니다. 비즈니스 로직(승인, 삭제 등)이 Slack API 응답을 기다리면 전체 처리가 블로킹됩니다.

해결

알림이 발생하면 메시지를 Redis List에 즉시 적재하고(`RPUSH`), 별도 워커가 1초마다 꺼내(`LPOP`) Slack API를 호출합니다.

```

비즈니스 로직 → RPUSH slack:notification:queue → 즉시 반환
SlackNotificationWorker → LPOP (1초 간격) → Slack API 호출
→ 실패 시 RPUSH 재삽입 (최대 3회)

```

비즈니스 로직은 Slack API 레이턴시 영향을 받지 않습니다. 워커가 순서대로 처리하므로 rate limit도 자연스럽게 지켜 집니다.

같은 패턴을 Infra 관련 알림에도 동일하게 적용합니다. 다만 큐(`slack:infra:notification:queue`)와 이를 소비하는 Worker(`InfraSlackNotificationWorker`)를 완전히 별도로 둡니다. 두 큐·두 Worker를 분리한 이유는 한쪽 알림이 밀리거나 실패해도 다른 쪽 발송에 영향을 주지 않게 하기 위해서입니다. 자세한 내용은 [Redis 키 카탈로그 4번](#)을 참고합니다.

적용 범위

rate limit이 있거나 응답이 느린 외부 API 호출을 비동기로 처리할 때 이 패턴을 사용합니다.

인증·보안

토큰을 왜 굳이 두 개로 나눴는지부터 설명하고, 거기서 로그인·재발급·로그아웃 흐름과 모든 요청을 가로채는 필터의 판단 기준까지 이어갑니다. 인증 코드는 한 군데만 잘못 고쳐도 전체 로그아웃되거나, 반대로 탈취된 세션이 계속 살아있는 사고로 이어질 수 있습니다. 그래서 "어떻게 동작하는가"보다 "왜 이렇게 만들었는가"를 먼저 알아야 안전하게 고칠 수 있습니다.

JWT와 Redis를 조합해 인증을 처리합니다. 여기서 AccessToken은 **AT**, RefreshToken은 **RT**로 줄여 씁니다. OAuth 스펙 용어는 아니고(스펙엔 `access_token` / `refresh_token` 으로 풀어 씀) 이 프로젝트에서만 쓰는 표현인데, RT는 실제로 코드에도 그대로 박혀 있습니다 — Redis 키를 `"RT:" + userId` 처럼 직접 만듭니다(`TokenService`, `UserLoginService`). AT는 Redis에 저장하지 않는 stateless 토큰이라 코드엔 등장하지 않고, RT와 짝을 맞추려고 문서에서만 쓰는 표현입니다.

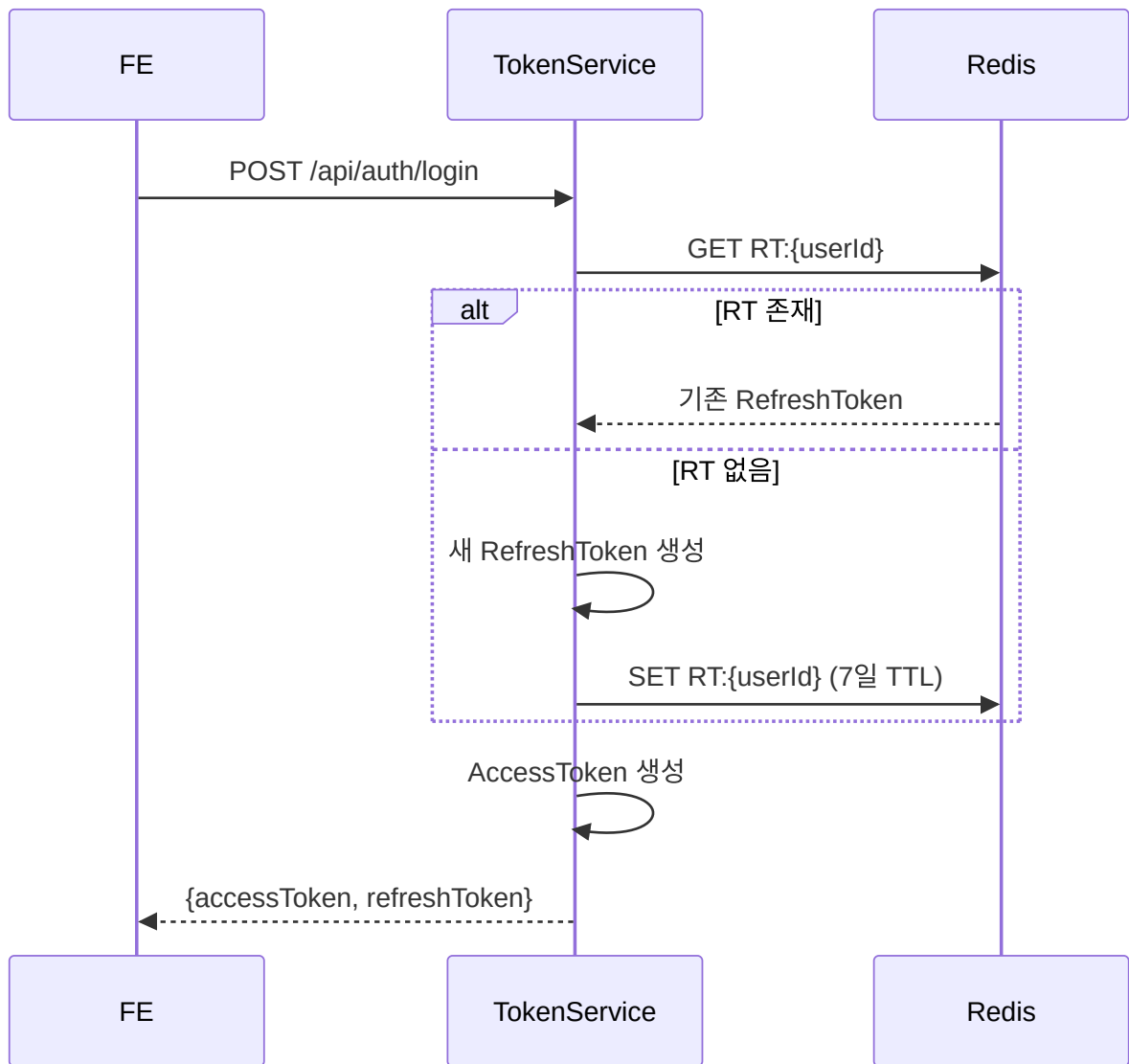
AT와 RT를 분리하는 이유는 **검증 속도**와 **세션 제어** 두 가지입니다. AT는 모든 API 요청에 붙어오기 때문에 Redis나 DB 조회 없이 서명만 검증해 빠르게 처리합니다(Stateless). 반면 RT는 Redis에 저장하기 때문에 언제든지 삭제해 즉시 세션을 종료할 수 있습니다. 로그아웃하면 Redis에서 RT를 지우고, 이후 AT가 만료돼 재발급을 시도할 때 Redis에 키가 없으므로 거부됩니다.

AccessToken	RefreshToken	TTL	application.yml 설정값 (기본 1시간)
application.yml 설정값 (기본 7일)	저장 위치	저장 안 함 (Stateless)	Redis RT:{userId}
검증 방식	서명만 확인, I/O 없음	Redis 조회 필요	사용 시점
모든 API 요청	AT 만료 시 재발급 한 번	세션 종료	만료 전 직접 취소 불가
Redis 키 삭제로 즉시 무효화			

토큰의 `subject` 에는 `userId` (Long)가 담깁니다. 서명 알고리즘은 HS256이며, secret key는 `application.yml` 의 `jwt.secret` 에서 읽어옵니다.

1. 토큰 발급 (로그인)

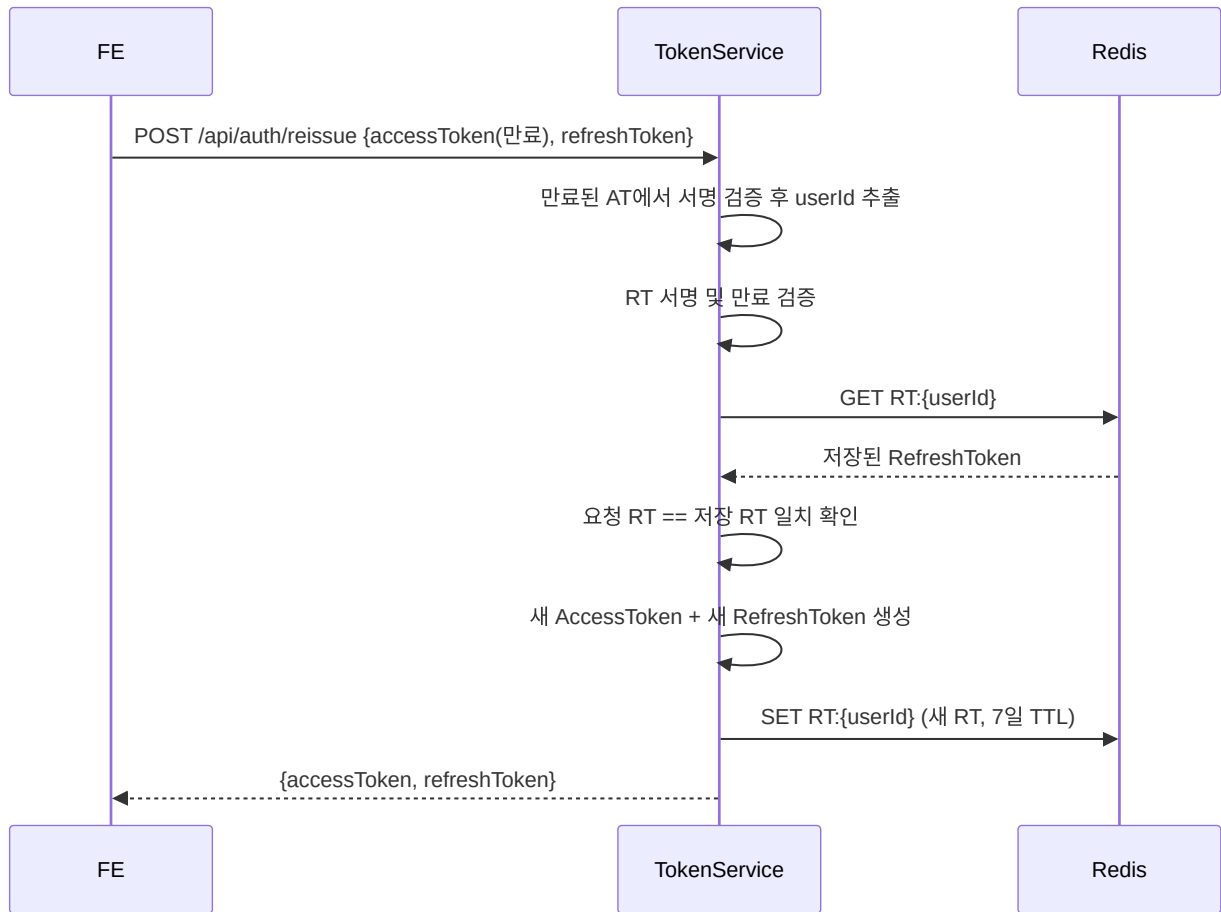
로그인 성공 시 AccessToken과 RefreshToken을 발급합니다. RefreshToken은 Redis에 이미 유효한 토큰이 있으면 재사용하고, 없으면 새로 발급합니다. 이 덕분에 여러 기기에서 로그인해도 RT가 중복 발급되지 않습니다.



2. 토큰 재발급

AT가 만료되면 클라이언트는 보관 중인 RT로 재발급을 요청합니다. 재발급 시 AT와 RT 모두 새로 발급됩니다. RT도 함께 교체하는 이유는 보안 때문입니다. RT가 탈취된 상황에서 정상 사용자가 먼저 재발급을 완료하면 Redis의 값이 새 RT로 교체되므로, 공격자가 이전 RT로 시도할 때 Redis 값과 일치하지 않아 거부됩니다.

재발급 흐름에서 주의할 점이 하나 있습니다. AT가 만료된 상태이므로 일반적인 검증(`parseClaimsJws`)을 쓰면 예외가 납니다. `JwtProvider`의 `getSubjectFromExpiredToken()`은 만료된 AT에서도 서명이 유효하면 `ExpiredJwtException`의 claims를 꺼내 `userId`를 반환합니다. 서명 자체가 잘못된 경우에는 예외가 그대로 전파돼 재발급이 거부됩니다.

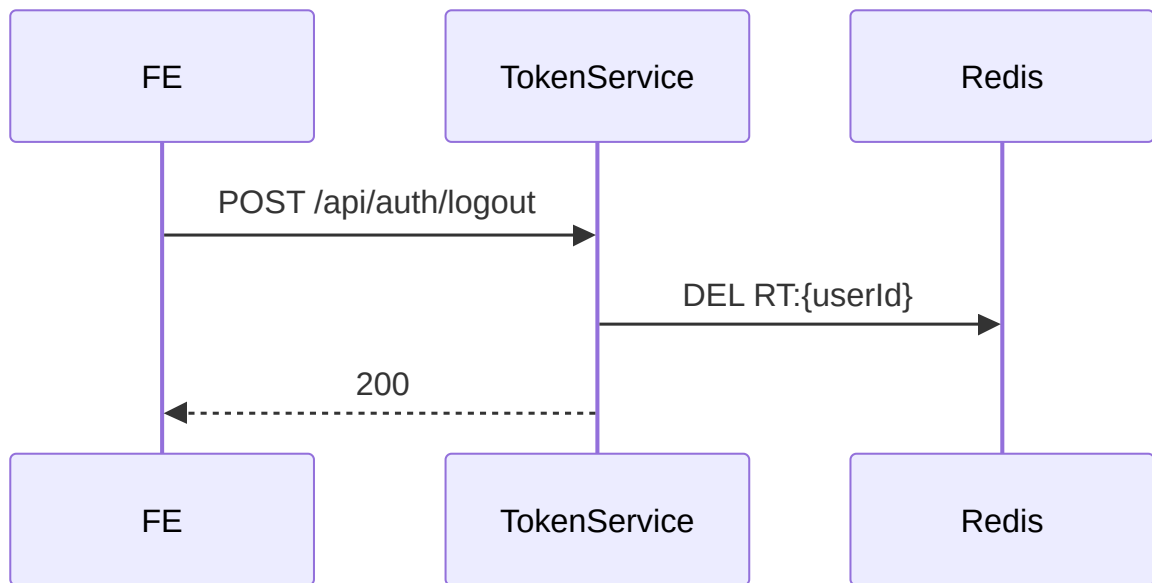


재발급이 거부되는 경우:

- 만료된 AT의 서명이 유효하지 않을 때
- RT가 만료됐을 때
- RT가 Redis에 없을 때 (로그아웃된 계정)
- 요청의 RT와 Redis에 저장된 RT가 다를 때

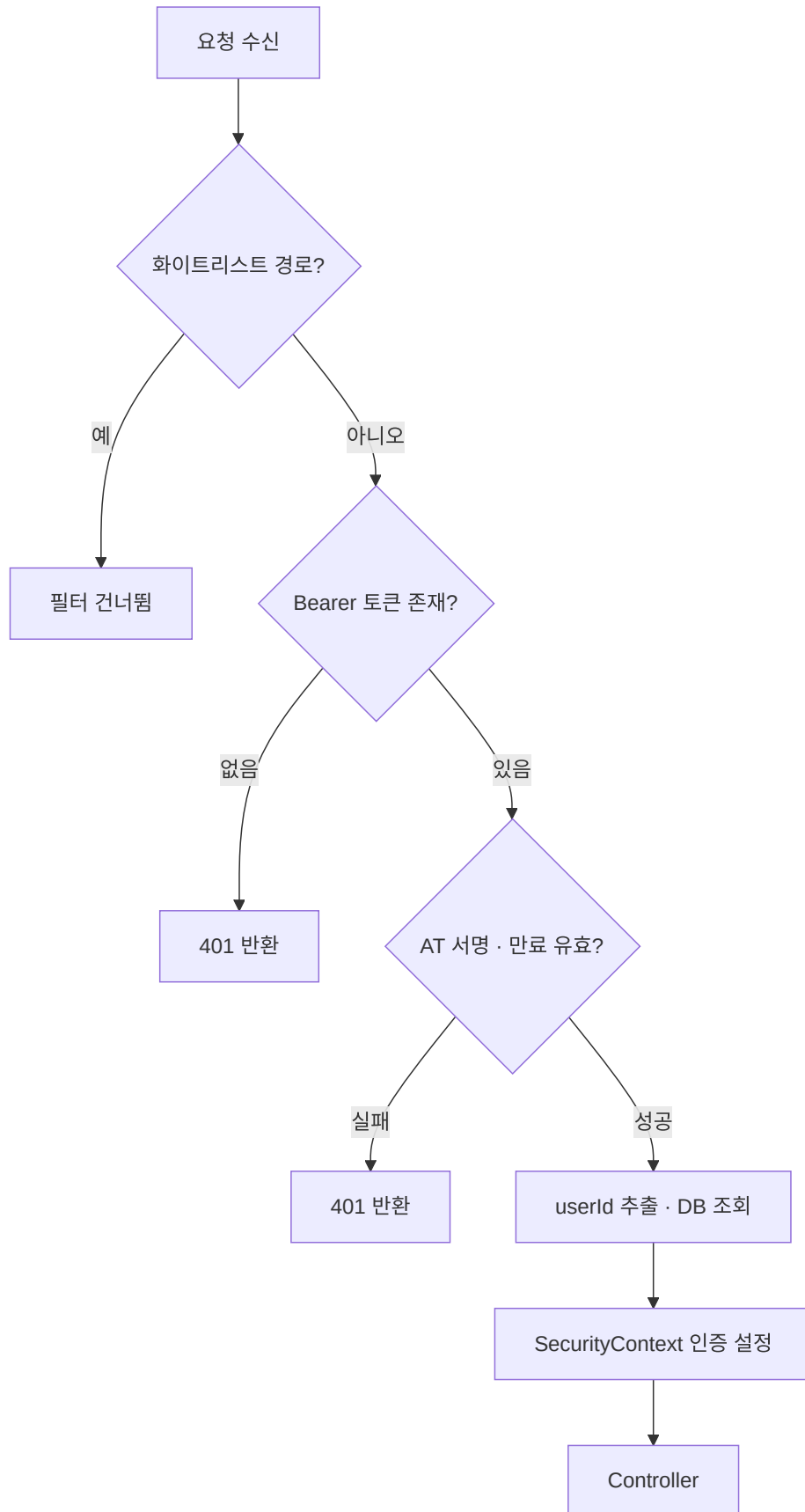
3. 로그아웃

Redis에서 `RT:{userId}` 키를 삭제합니다. 이후 해당 `userId`의 RT로 재발급 요청이 들어오면 Redis에 키가 없어 거부됩니다. AccessToken은 서버에 저장하지 않으므로 만료 전까지는 유효하지만, 짧은 TTL(1시간) 덕분에 실제 보안 영향은 제한됩니다.



4. 요청 인증 흐름 (JwtAuthenticationFilter)

모든 API 요청은 `JwtAuthenticationFilter` 를 통과합니다. `OncePerRequestFilter` 를 상속하므로 요청당 정확히 한 번만 실행됩니다.



필터가 건너뛰는 화이트리스트 경로(`UNPROTECTED_PATHS`)는 [운영 가이드 8절\(화이트리스트 설정\)](#)에서 관리합니다.

AccessToken은 `Authorization: Bearer {token}` 헤더에서 추출합니다. 헤더가 없거나 `Bearer` 로 시작하지 않으면 즉시 401을 반환합니다.

검증 실패 시 필터에서 바로 JSON 에러 응답을 씁니다. Spring의 `ExceptionHandlerFilter` 를 거치지 않기 때문에 `GlobalExceptionHandler` 가 아닌 필터 내부의 `setErrorResponse()` 에서 직접 응답을 만듭니다.

5. 역할 기반 접근 제어

사용자 `Role` 은 `USER` 와 `ADMIN` 두 가지입니다. 관리자 기능은 `/api/admin/**` 경로로 분리되어 있으며, `SecurityConfig` 에서 해당 경로에 `ADMIN` 권한을 요구합니다.

`Role` 을 변경하는 별도 API는 제공하지 않습니다. 필요한 경우 MySQL에 직접 접속해 수정합니다. 접속 방법은 [운영 가이드 7절\(MySQL 직접 접속\)](#)을 참고합니다.

외부 연동

BE는 계정을 만들거나 컨테이너를 띄우는 실제 작업을 스스로 하지 않고 Infra Server라는 바깥 시스템에 맡깁니다. 그 경계 너머에서 무슨 일이 벌어지는지, 그리고 경계 너머가 실패했을 때 BE가 자기 상태를 어떻게 지키는지를 다룹니다. "승인을 눌렀는데 왜 계정이 안 만들어지지", "Pod는 생겼는데 왜 신청이 PENDING으로 돌아갔지" — 이런 질문의 답은 대부분 여기 있습니다.

BE는 Ubuntu 계정·Pod 생성·삭제를 직접 구현된 Infra Server에 위임합니다. 두 종류의 작업은 타임아웃 특성이 달라 WebClient 인스턴스를 분리해 관리합니다.

WebClient	용도	타임아웃
<code>configWebClient</code>	Ubuntu 계정 생성·삭제	응답 120s(<code>config.timeout-seconds</code>), 연결 10s
<code>podWebClient</code>	Pod 생성·삭제	응답 300s(<code>config.pod-timeout-seconds</code>), 연결 30s

Bean 정의는 `WebClientConfig` 에 있습니다.

Pod 생성에 긴 타임아웃을 두는 이유는 Docker 이미지를 처음 pull할 때 시간이 크게 걸릴 수 있기 때문입니다. 두 WebClient 모두 같은 `config.base-url` (Infra Server)을 바라봅니다.

1. Ubuntu 계정 관리 (configWebClient)

계정 생성

관리자 승인 시 `AdminRequestCommandService` 에서 호출합니다. 생성 성공 시 Infra Server가 반환한 `uid` 와 `gid` 를 Request에 저장합니다.

요청


```
PUT /accounts/users
Content-Type: application/json
```

```
{
  "name": "ubuntu_username",
  "passwd_base64": "base64로 인코딩된 초기 비밀번호",
  "gecos": "사용자 실명",
  "primary_group_name": "ubuntu_username",
  "supplementary_groups": [
    { "name": "ailab", "gid": 2001 }
  ]
}
```

필수 필드는 `name` 과 `passwd_base64` 이며, 나머지는 선택입니다. `supplementary_groups` 는 사용자를 추가 그룹에 소속시킬 때 사용합니다.

응답 (HTTP 201)

Swagger 스펙에 응답 스키마가 명시되어 있지 않으나, BE는 응답 본문에서 `uid` 와 `gid` 를 읽어 Request에 저장합니다.

계정 삭제

`UbuntuAccountService.deleteUbuntuAccount()` 에서 호출합니다. 만료 처리 스케줄러와 보상 트랜잭션 두 경로에서 호출됩니다.

```
DELETE /accounts/users/{username}
```

404 응답은 계정이 이미 없는 것이므로 정상 처리로 간주하고 넘어갑니다. 이미 지워진 계정을 다시 삭제 요청해도 결과는 똑같이 "계정 없음"이라, 같은 요청을 여러 번 보내도 안전합니다. 그래서 보상 트랜잭션 경로에서 계정이 애초에 생성되지 않았을 때도 걱정 없이 호출할 수 있습니다. 400이나 다른 에러 코드는

`BusinessException(UBUNTU_USER_DELETION_FAILED)` 으로 변환됩니다.

2. Pod 관리 (podWebClient)

Pod 생성

승인 처리 중 Ubuntu 계정 생성 성공 후 호출합니다. 응답에서 `podName`, `node`, `ports` 정보를 받아 `PodExternalPort` 테이블에 저장합니다.

요청

```
POST /create-pod
Content-Type: application/json
```

```
{
  "username": "ubuntu_username"
}
```

응답 (HTTP 201)

```
{
  "status": "created",
  "node": "farm1",
  "pod_name": "ai lab-user2100-1",
  "ports": [
    { "usage_purpose": "ssh", "internal_port": 22, "external_port": 30022 },
    { "usage_purpose": "jupyter", "internal_port": 8888, "external_port": 30888 }
  ]
}
```

4xx/5xx 응답이 오거나 응답 본문의 `pod_name` 이 null이면 `BusinessException(POD_CREATION_FAILED)` 를 던집니다. 이 예외는 보상 트랜잭션을 트리거해 앞서 생성한 Ubuntu 계정을 삭제하고 Request 상태를 PENDING으로 되돌립니다.

Pod 삭제

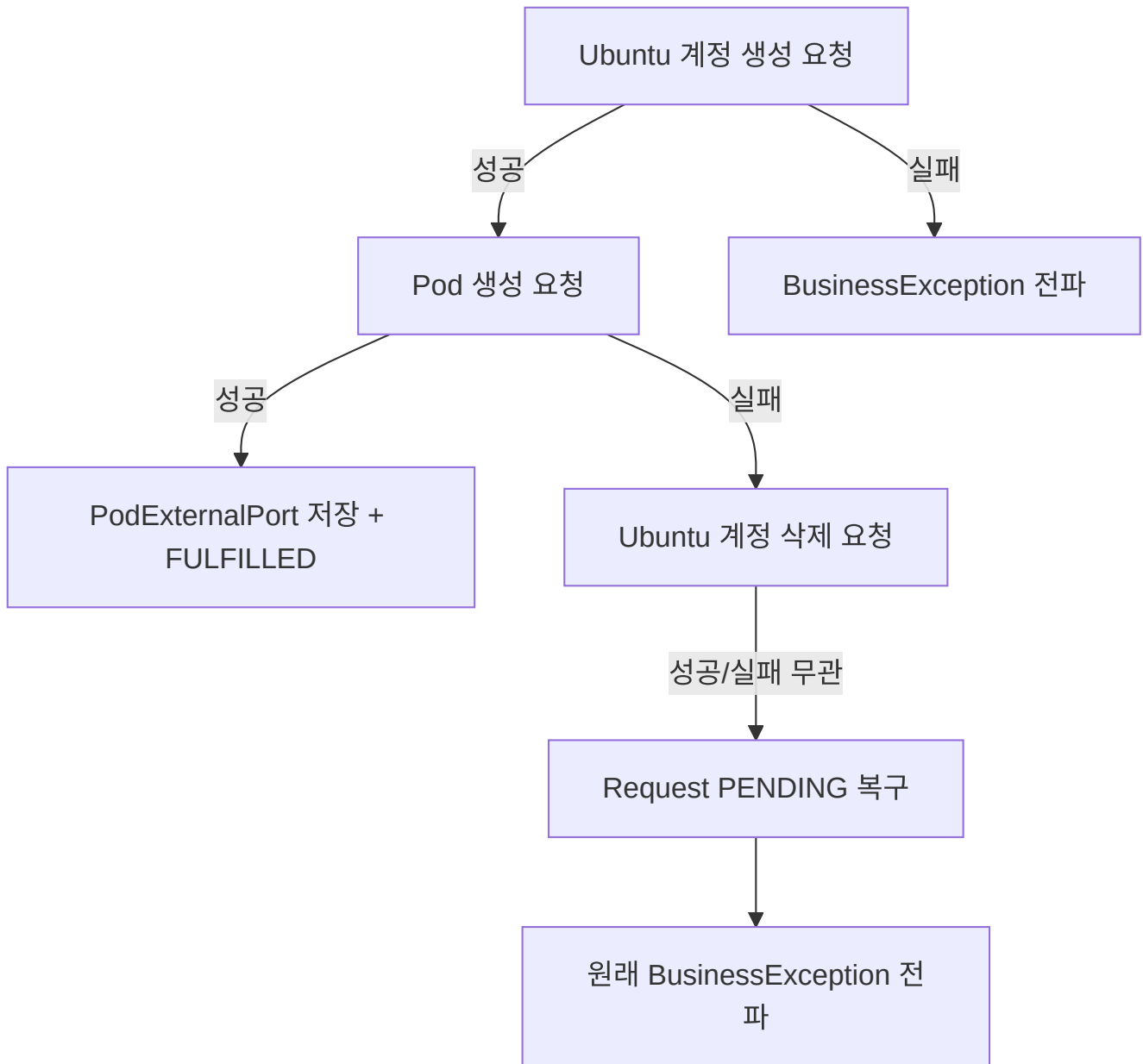
```
POST /delete-pod
Content-Type: application/json
```

```
{
  "pod_name": "ai lab-user2100-1"
}
```

`pod_name` 이 null이면 호출 자체를 건너뛰니다(이미 Pod가 없는 경우를 대비). 400이나 500 응답은 `BusinessException` 으로 변환됩니다.

3. 에러 처리와 보상 트랜잭션 연계

Ubuntu 계정 생성 → Pod 생성 순서로 진행하므로, 중간에 실패하면 이전 단계를 되돌려야 합니다.



Pod 생성 실패 시 Ubuntu 계정 삭제(`deleteUbuntuAccount`)가 추가로 실패하더라도 원래 예외(`POD_CREATION_FAILED`)가 그대로 전파됩니다. 로그에는 두 실패 모두 남습니다.

4. Prometheus 연동

FE 대시보드가 `GET /api/monitoring/metrics`를 폴링하면, BE가 Prometheus HTTP API에 PromQL 쿼리를 날려 결과를 가공해 반환합니다. 이 엔드포인트는 인증 없이 접근 가능한 화이트리스트 경로(`/api/monitoring/**`)입니다.

Prometheus 서버 주소는 `application.yml`의 `prometheus.base-url`에서 읽으며, 기본값은 클러스터 내부 주소(`http://10.98.198.241:9090`)입니다. 각 쿼리는 5초 타임아웃이 적용되고, 실패 시 예외를 전파하지 않고 빈 값을 반환합니다. Prometheus가 일시적으로 내려가도 대시보드 API 자체는 200으로 응답합니다.

실행하는 PromQL 쿼리

쿼리	설명
<code>avg by(Hostname)(DCGM_FI_DEV_GPU_UTIL)</code>	서버별 평균 GPU 사용률 (%)
<code>count by(Hostname)(DCGM_FI_DEV_GPU_UTIL)</code>	서버별 GPU 개수
<code>sum by(cluster)(cluster_monitor_container_running)</code>	클러스터별 활성 컨테이너 수

GPU 메트릭은 DCGM(NVIDIA Data Center GPU Manager) Exporter가 수집합니다. 활성 컨테이너 수는 별도로 구현된 `cluster_monitor_container_running` 메트릭에서 읽어옵니다.

응답 구조

```
{
  "gpuServers": [
    { "hostname": "farm1", "gpuUtil": 43.2, "gpuCount": 4 },
    { "hostname": "lab1", "gpuUtil": 12.5, "gpuCount": 2 }
  ],
  "activeContainers": {
    "farm": 8,
    "lab": 3
  }
}
```

`gpuUtil` 은 소수점 한 자리로 반올림해서 내려줍니다. 서버 목록은 `hostname` 기준 알파벳 순으로 정렬됩니다.

Redis 키 카탈로그

Redis는 코드를 안 보면 지금 이 키가 왜 이 값을 갖고 있는지 짐작하기 어렵습니다. 장애 상황에서 코드를 뒤질 필요 없이 "이 키가 뭘 의미하고 지워도 되는지, 왜 이 TTL인지"를 아래 표만 보고 판단할 수 있게 정리했습니다. 장애 대응·디버깅은 물론 새 캐시나 중복 방지 키를 설계할 때도 여기를 기준으로 삼습니다.

전체 요약

분류	키 패턴	타입	TTL
인증	<code>email:verify:{email}</code>	String	5분
인증	<code>VERIFIED:{email}</code>	String	10분
세션	<code>RT:{userId}</code>	String	7일

분류	키 패턴	타입	TTL
이메일 중복 방지	<code>email:preexpiry:{requestId}:{dayLabel}:{date}</code>	String	25시간
Slack 큐	<code>slack:notification:queue</code>	List	없음
Slack 큐	<code>slack:infra:notification:queue</code>	List	없음
Slack 캐시	<code>slack:cache:users:list</code>	String	1시간

1. 인증 키

이메일 인증 흐름에서 사용하는 두 개의 키입니다. 둘 다 TTL이 지나면 자동으로 사라지고, 정상 경로에서는 완료 즉시 명시 삭제됩니다.

`email:verify:{email}`

이메일 인증번호를 임시 저장합니다.

저장 POST /api/auth/email/send → SET email:verify:user@example.com = "183726" (TTL 5분)
 확인 POST /api/auth/email/verify → GET email:verify:user@example.com
 삭제 인증 성공 직후 → DEL email:verify:user@example.com
 (5분 안에 인증 안 하면 TTL 만료로 자동 삭제)

항목	값
값 형태	숫자 인증번호 문자열 (ex: <code>"183726"</code>)
TTL	5분 (<code>AUTH_CODE_EXPIRE_SECONDS = 300</code>)

`VERIFIED:{email}`

인증번호 확인이 완료됐음을 나타내는 상태 키입니다. 이 키가 없으면 회원가입 API가 400을 반환합니다.

저장 인증번호 일치 확인 후 → SET VERIFIED:user@example.com = "true" (TTL 10분)
 삭제 회원가입 완료 직후 → DEL VERIFIED:user@example.com
 (10분 안에 가입 안 하면 TTL 만료로 자동 삭제)

항목	값
값 형태	<code>"true"</code>

항목	값
TTL	10분

TTL 설계 이유: 인증번호 확인(5분)과 회원가입 폼 제출(10분)을 단계별로 분리한 이유는, 사용자가 인증 직후 바로 가입 폼을 제출하지 않을 수 있기 때문입니다. 더 짧은 TTL을 가진 `email:verify` 키를 먼저 삭제하고, `VERIFIED` 키에는 5분 더 긴 TTL(10분)을 줍니다.

흐름 요약



2. 세션 키

`RT:{userId}`

Refresh Token을 저장하는 핵심 세션 키입니다. 이 키가 없으면 해당 사용자는 로그아웃 상태입니다.

저장 로그인 성공	→ SET RT:42 = "<JWT>" (TTL 7일)
갱신 토큰 재발급	→ SET RT:42 = "<새 JWT>" (TTL 7일, 기존 키 재사용 가능)
삭제 로그아웃	→ DEL RT:42
(7일 미갱신 시 TTL 만료로 자동 삭제 — 자동 로그아웃)	

항목	값
값 형태	Refresh Token JWT 문자열
TTL	7일 (<code>jwt.refresh-token-expire-time</code>)
<code>{userId}</code>	User 테이블의 PK (<code>BIGINT</code>)

설계 포인트: Access Token은 Redis에 저장하지 않습니다(Stateless). Refresh Token만 Redis에 저장해서, 필요시 `DEL RT:{userId}` 로 즉시 세션을 무효화(강제 로그아웃)할 수 있습니다. 자세한 내용은 [인증·보안](#)을 참고합니다.

확인 방법

```
# 사용자 42번이 로그인 상태인지
EXISTS RT:42      # 1 = 로그인, 0 = 로그아웃

# 남은 세션 시간 (초)
TTL RT:42        # -2 = 만료됨, 양수 = 남은 초

# 현재 로그인된 사용자 전체 조회
KEYS RT:*
```

3. 이메일 중복 방지 키

email:preexpiry:{requestId}:{dayLabel}:{date}

스케줄러가 같은 날 같은 요청에 만료 예고 이메일을 두 번 보내지 않도록 막습니다.

키 예시: email:preexpiry:42:7일:2026-07-15

체크 이메일 발송 직전 → SETNX email:preexpiry:42:7일:2026-07-15 "sent" (TTL 25h)
키가 이미 있으면 → 발송 건너뛴
키가 없으면 → 발송 진행 후 키 생성
삭제 TTL 만료 (25시간 후 자동)

항목	값
값 형태	"sent"
TTL	25시간 (24시간 주기보다 1시간 길게 — 스케줄러 실행 시각 편차 흡수)
{dayLabel}	7일, 3일, 1일 (만료까지 남은 날 수)
{date}	만료 목표 날짜 yyyy-MM-dd (today+7/3/1 중 해당일, 발송 당일이 아님)

Fail-open 설계: Redis 장애 시 `setIfAbsent()` 가 예외를 던집니다. 이 경우 `false`를 반환해 이메일 발송을 허용합니다(fail-open). 중복 발송보다 누락이 더 나쁘다는 판단입니다. 자세한 패턴은 [핵심 설계 패턴](#) 3번을 참고합니다.

확인 방법

```
# 특정 요청의 중복 방지 키 전체 조회
KEYS email:preexpiry:42:*

# 오늘자 전체 만료 예고 키 조회
KEYS email:preexpiry:*:2026-07-15
```

4. Slack 알림 큐

Slack Webhook API는 채널당 초당 1건의 rate limit이 있습니다. 비즈니스 로직이 직접 Slack API를 호출하면 블로킹됩니다. 대신 Redis List를 큐로 사용해 비동기 처리합니다.

공통 구조

알림 발생 시 → RPush (오른쪽 삽입, FIFO)
전용 Worker → LPop (왼쪽 꺼내기, 1초 간격)
발송 실패 → RPush로 재삽입 (최대 3회, 3회 초과 시 폐기)

두 큐는 소비하는 Worker 클래스도 분리되어 있습니다. `slack:notification:queue`는 `SlackNotificationWorker`가, `slack:infra:notification:queue`는 `InfraSlackNotificationWorker`가 각각 전담합니다. 서로 완전히 독립 운영되므로 한쪽 큐가 밀려도 다른 쪽 발송에는 영향이 없습니다.

`slack:notification:queue`

일반 비즈니스 알림 큐입니다 (신청 접수, 승인, 거절, 삭제 등). `SlackNotificationWorker`가 소비합니다.

항목	값
값 형태	<code>SlackNotificationDTO</code> JSON 직렬화
TTL	없음 (영구 보존, 꺼낼 때까지 유지)

`slack:infra:notification:queue`

Infra 서버 관련 알림 전용 큐입니다. `slack:notification:queue`와 별도로 운영해 우선순위나 처리 경로를 분리합니다. `InfraSlackNotificationWorker`가 소비합니다.

항목	값
값 형태	<code>slack:notification:queue</code> 와 동일

확인 방법

```
# 현재 큐에 쌓인 메시지 수
LLEN slack:notification:queue
LLEN slack:infra:notification:queue

# 큐 내용 조회 (삭제하지 않고 확인)
LRANGE slack:notification:queue 0 -1
LRANGE slack:infra:notification:queue 0 -1
```


큐 길이가 계속 늘어난다면 `SlackNotificationWorker` 가 멈췄거나 Slack API 장애를 의심합니다. 자세한 대응은 [운영 가이드 11절](#)을 참고합니다.

동작을 직접 확인하고 싶을 때

메시지를 수동으로 넣어보고 Worker가 가져가는 과정을 실시간으로 볼 수 있습니다. DTO 필드는 실제 `SlackNotificationDTO` 와 맞춰야 역직렬화 예러가 나지 않습니다.

```
# 큐에 테스트 메시지 하나 삽입 (RPUSH는 오른쪽 끝에 추가)
RPUSH slack:notification:queue '{"type":"DM","username":"test_user","email":"test@test.com","message":"테스트"}'
# 결과가 (integer) 1이면 큐에 1건 쌓인 것

# 별도 터미널에서 실시간 명령 로그 확인
MONITOR

# 이 상태에서 스케줄러나 알림을 트리거하면 RPUSH → LPOP이 순서대로 찍히는 것을 볼 수 있음
```

5. Slack 캐시

`slack:cache:users:list`

Slack 워크스페이스 멤버 목록을 캐싱합니다. DM 발송 시 이메일 → Slack UserId 변환에 사용됩니다. 매번 Slack `users.list` API를 호출하면 rate limit에 걸릴 수 있어 1시간 TTL로 캐싱합니다.

```
저장 Slack users.list API 호출 후 → SET slack:cache:users:list = <멤버 목록 JSON> (TTL 1h)
삭제 TTL 만료 (1시간 후 자동)
```

항목	값
값 형태	Slack 멤버 목록 JSON 직렬화
TTL	1시간

redis-cli 빠른 참조

—— Redis Pod 접속

```
kubectl exec -it <redis-pod-name> -n <namespace> -- redis-cli
```

—— 키 존재 여부 / 값 / TTL ——

EXISTS RT:42 # 1 = 있음, 0 = 없음

GET RT:42 # 값 조회

TTL RT:42 # 남은 TTL (초). -2 = 만료/없음

—— 인증 관련

EXISTS VERIFIED:user@example.com # 인증 완료 상태 확인

KEYS email:verify:* # 인증번호 대기 중인 이메일 목록

—— Slack 큐

LLEN slack:notification:queue # 큐 메시지 수

LRANGE slack:notification:queue 0 4 # 앞에서 5개 확인 (삭제 안 함)

—— 만료 예고 중복 방지 ——

KEYS email:preexpiry:42:* # requestId=42의 발송 이력

KEYS email:preexpiry:* # 전체 발송 이력 (주의: 많으면 느림)

—— 운영 환경 전체 키 스캔 (KEYS * 대신 사용) ——

```
SCAN 0 MATCH * COUNT 100
```

KEYS * 명령은 Redis를 블로킹합니다. 운영 환경에서는 반드시 **SCAN** 을 사용하세요.

에러 코드 카탈로그

API가 실패했을 때 클라이언트가 받는 에러 메시지와 HTTP 상태 코드는 전부 **ErrorCode** enum 한 곳에서 정의합니다. "이 에러가 무슨 뜻이고 어떤 상황에서 나는지"를 코드를 열어보지 않고 여기서 바로 찾을 수 있습니다. FE가 특정 에러 메시지를 받고 원인을 추적할 때, 또는 새 에러 케이스를 추가하기 전 비슷한 케이스가 이미 있는지 확인할 때 먼저 봅니다.

전체 59개 케이스가 있으며, **ErrorCode** 에 정의된 그룹 순서 그대로 정리했습니다. 새 에러를 추가할 때는 관련된 그룹 (예: Group Error, Pod Error) 안에 추가하고, 마땅한 그룹이 없다면 새 그룹 주석을 만듭니다.

위치: **DGU_AI_LAB.admin.be.error.ErrorCode**

HTTP 상태 코드 공통 에러

코드	상태	메시지
BAD_REQUEST	400	잘못된 요청입니다.
INVALID_INPUT_VALUE	400	입력값이 올바르지 않습니다.
MISSING_REQUEST_PARAMETER	400	필수 요청 파라미터가 누락되었습니다.
UNAUTHORIZED	401	리소스 접근 권한이 없습니다.
INVALID_ACCESS_TOKEN	401	액세스 토큰의 형식이 올바르지 않습니다. Bearer 타입을 확인해 주세요.
INVALID_ACCESS_TOKEN_VALUE	401	액세스 토큰의 값이 올바르지 않습니다.
EXPIRED_ACCESS_TOKEN	401	액세스 토큰이 만료되었습니다. 재발급 받아주세요.
INVALID_REFRESH_TOKEN	401	리프레시 토큰의 형식이 올바르지 않습니다.
INVALID_REFRESH_TOKEN_VALUE	401	리프레시 토큰의 값이 올바르지 않습니다.
EXPIRED_REFRESH_TOKEN	401	리프레시 토큰이 만료되었습니다. 다시 로그인해 주세요.
NOT_MATCH_REFRESH_TOKEN	401	일치하지 않는 리프레시 토큰입니다.
FORBIDDEN	403	리소스 접근 권한이 없습니다.
ACCESS_DENIED	403	접근이 거부되었습니다.
RESOURCE_NOT_FOUND	404	해당 리소스를 찾을 수 없습니다.
ENTITY_NOT_FOUND	404	엔티티를 찾을 수 없습니다.
METHOD_NOT_ALLOWED	405	잘못된 HTTP method 요청입니다.
CONFLICT	409	이미 존재하는 리소스입니다.
FILE_SIZE_EXCEEDED	413	이미지 최대 크기를 초과하였습니다.
INTERNAL_SERVER_ERROR	500	서버 내부 오류입니다.
FILE_UPLOAD_FAILED	500	파일 업로드에 실패하였습니다.
JSON_PARSING_FAILED	500	JSON 파싱에 실패하였습니다.
SLACK_DM_CHANNEL_FAILED	502	Slack DM 채널 열기를 실패하였습니다.
EXTERNAL_API_ERROR	502	외부 API 호출에 실패했습니다.
SLACK_SEND_FAILED	503	Slack 메시지 전송에 실패하였습니다.

이 그룹은 특정 도메인에 묶이지 않는 범용 에러입니다. 도메인 로직에서 던질 에러가 마땅치 않을 때가 아니라면, 아래 도메인별 에러를 우선 사용합니다.

사용자(User) 에러

코드	상태	메시지
EMAIL_NOT_VERIFIED	401	이메일 인증이 완료되지 않았습니다.
AUTHENTICATION_FAILED	401	사용자 인증에 실패하였습니다.
USER_NOT_FOUND	404	사용자를 찾을 수 없습니다.
USER_ALREADY_EXISTS	409	이미 가입된 사용자입니다.
USER_ALREADY_ACTIVE	409	이미 활성화된 사용자입니다.
DUPLICATE_NAME	409	중복된 닉네임입니다.
INVALID_LOGIN_INFO	400	잘못된 로그인 입력값입니다.
INVALID_AUTH_CODE	400	올바르지 않은 인증 코드입니다.
GROUP_NOT_FOUND	404	지정된 그룹을 찾을 수 없습니다.
UID_ALLOCATION_FAILED	502	외부 API 응답에서 UID/GID를 확인할 수 없습니다.
DUPLICATE_USERNAME	409	이미 사용하고 있는 username입니다. 같은 사용자이더라도 다른 username을 입력해주세요.
ACCOUNT_DISABLED	404	비활성화된 유저입니다. 관리자에게 문의하세요.
SLACK_USER_NOT_FOUND	404	Slack 사용자를 찾을 수 없습니다.
SLACK_USER_EMAIL_NOT_MATCH	404	이메일이 일치하는 Slack 사용자를 찾을 수 없습니다.
INVALID_PASSWORD	401	현재 비밀번호가 일치하지 않습니다.
PASSWORD_CHANGE_SAME_AS_OLD	400	새 비밀번호가 현재 비밀번호와 동일합니다.

그룹(Group) 에러

코드	상태	메시지
NO_AVAILABLE_GROUPS	404	존재하는 그룹 정보가 없습니다.
DUPLICATE_GROUP_ID	409	외부 API: 그룹명 또는 GID 충돌
DUPLICATE_GROUP_NAME	409	중복된 그룹 이름입니다.
GROUP_CREATION_FAILED	502	외부 API: 필수 필드 누락 또는 형식 오류

코드	상태	메시지
GID_ALLOCATION_FAILED	502	외부 API 응답에서 GID를 확인할 수 없습니다.
FORBIDDEN_REQUEST	400	요청된 우분투 사용자 이름은 로그인한 사용자의 계정이 아닙니다.
INVALID_GROUP_MEMBER	400	존재하지 않는 사용자입니다.

DUPLICATE_GROUP_ID , GROUP_CREATION_FAILED 처럼 "외부 API:"로 시작하는 메시지는 Infra Server 응답을 그대로 반영한 것입니다. Infra Server 쪽 원인까지 확인하려면 [외부 연동](#)을 참고합니다.

승인(Approval) 에러

코드	상태	메시지
USER_APPROVAL_NOT_FOUND	404	해당 유저의 승인 정보를 찾을 수 없습니다.

리소스 그룹(ResourceGroup) 에러

코드	상태	메시지
RESOURCE_GROUP_NOT_FOUND	404	리소스 그룹을 찾을 수 없습니다.
NO_AVAILABLE_RESOURCES	404	사용 가능한 리소스가 없습니다.

노드(Node) 에러

코드	상태	메시지
NODE_NOT_FOUND	404	노드를 찾을 수 없습니다.

신청(Request) 에러

코드	상태	메시지
INVALID_REQUEST_STATUS	409	이미 처리된 신청입니다.
UNSUPPORTED_CHANGE_TYPE	400	지원되지 않는 요청 타입(enum)입니다.

INVALID_REQUEST_STATUS 는 이미 승인/거절 처리된 Request를 다시 승인·거절하려 할 때, 또는 PROCESSING 상태인 Request에 중복 요청이 들어왔을 때 발생합니다. [시스템 아키텍처](#) 1번의 상태 전이표와 함께 보면 어느 전이에서 막힌 것인지 파악하기 쉽습니다.

계정(Account) 에러

코드	상태	메시지
UBUNTU_USER_DELETION_FAILED	502	우분투 계정 삭제에 실패했습니다.
USER_CREATION_FAILED	502	사용자 계정 생성에 실패했습니다.
INVALID_USERNAME_FORMAT	400	잘못된 사용자명 형식입니다.

Pod 에러

코드	상태	메시지
POD_CREATION_FAILED	502	Pod 생성 API 요청에 실패했습니다.
POD_DELETION_FAILED	502	Pod 삭제 API 요청에 실패했습니다.

`POD_CREATION_FAILED` 는 3단 보상 트랜잭션을 트리거하는 대표적인 에러입니다. 발생 시 Ubuntu 계정이 자동으로 정리 되고 Request가 PENDING으로 복구됩니다. 흐름은 [핵심 설계 패턴](#) 4번을 참고합니다.